

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Systems and Methods for Big and Unstructured Data Project

Author(s): **Francesco Riccardi (10651584)**

Alessandro Romito (10661916)

Marco Luca Previtera (10682248)

Simone Lo Gatto (10736180)

Lorenzo Ferretti(10713719)

Group Number: **9**

Academic Year: 2022-2023

Contents

Contents **i**

I First Delivery: Neo4J **1**

1 Introduction **3**
1.1 Problem Specification . . . 3
1.2 Hypothesis . . . 3

2 ER Diagram **5**
2.1 Entities and Relationships . . . 5
2.2 ER Diagram description . . . 5

3 Graph Diagram **7**
3.1 Graph Diagram description . . . 7
3.2 ER Diagram and Neo4j differences . . . 9

4 Dataset Description and processing **11**

5 Queries and Commands **17**
5.1 Query 1 . . . 17
5.2 Query 2 . . . 18
5.3 Query 3 . . . 19
5.4 Query 4 . . . 20
5.5 Query 5 . . . 21
5.6 Query 6 . . . 22
5.7 Query 7 . . . 23
5.8 Query 8 . . . 24
5.9 Query 9 . . . 25
5.10 Query 10 . . . 26

5.11 Query 11 . . . 27
5.12 Query 12 . . . 27
5.13 Query 13 . . . 28
5.14 Query 14 . . . 29
5.15 Command 1 . . . 29
5.16 Command 2 . . . 30
5.17 Command 3 . . . 31
5.18 Command 4 . . . 31
5.19 Extra Commands . . . 32
5.19.1 Import in the Database . . . 32
5.19.2 Creation Coauthor relationship . . . 33
5.19.3 Creation Cite relationship . . . 33
5.19.4 Create a relationship that associate the author to his past University 34

II Second Delivery: MongoDB 35

6 Introduction 37
6.1 Specification . . . 37
6.2 Hypothesis and assumptions . . . 37

7 Document Description 39
7.1 Document schema . . . 39

8 Data Creation and Transformation 43
8.1 Pre-processing . . . 43
8.2 Upload Process . . . 45

9 Queries and Commands 47
9.1 Command 1 . . . 47
9.2 Command 2 . . . 47
9.3 Command 3 . . . 52
9.4 Command 4 . . . 53
9.5 Command 5 . . . 53
9.6 Command 6 . . . 53
9.7 Command 7 . . . 53
9.8 Query 1 . . . 54
9.9 Query 2 . . . 56
9.10 Query 3 . . . 57

9.11 Query 4 . . . 58
9.12 Query 5 . . . 59
9.13 Query 6 . . . 60
9.14 Query 7 . . . 63
9.15 Query 8 . . . 63
9.16 Query 9 . . . 65
9.17 Query 10 . . . 66
9.18 Query 11 . . . 68
9.19 Query 12 . . . 69
9.20 Query 13 . . . 70
9.21 Query 14 . . . 72
9.22 Query 15 . . . 73
9.23 Query 16 . . . 75

III Third Delivery: Spark 79

10 Introduction 81
10.1 Specification . . . 81
10.2 Hypothesis and assumptions . . . 81

11 Document Description 83
11.1 Document schema . . . 83

12 Data Creation and Transformation 85
12.1 Pre-processing . . . 85
12.2 Upload Process . . . 85

13 Queries and Commands 87
13.1 Command 1 . . . 87
13.2 Command 2 . . . 89
13.3 Command 3 . . . 90
13.4 Command 4 . . . 91
13.5 Command 5 . . . 92
13.6 Query 1 . . . 93
13.7 Query 2 . . . 94
13.8 Query 3 . . . 95
13.9 Query 4 . . . 95
13.10Query 5 . . . 97

13.11Query 6 98
13.12Query 7 98
13.13Query 8 99
13.14Query 9 100
13.15Query 10 102
13.16Query 11 (surplus) 102

Part I

First Delivery: Neo4J

1 | Introduction

1.1. Problem Specification

The aim of this assignment is to design a 'query graph data structure' in Neo4j for supporting a bibliographic Database. The database must store a large scale Dataset recording publications with their features and their descriptions in order to have an overview of the library.

1.2. Hypothesis

The assumptions taken into account are the following:

- All data are assumed to be truthful without further verification;
- We are using the concept of "publication" referring to Book, Article, Thesis, Online Resource(Www), Proceeding, Inproceeding and Incollection, instead, Journal isn't belonging to this family of Entities;
- Whoever uploads the data relating to publications does so by entering all the information in a correct way;
- An author doesn't insert the same work more than once so that redundancy and duplicates are avoided in the system;
- The Orcid is a way to identify the researchers and it is mandatory, but in reality is not;
- Every publication has an associated dynamic attribute "CitationScore" that computes the number of works that quote a specific one. Every author has his own "CitationScore" attribute too, it is computed with a formula which aims to balance the score between the authors. As said, this attribute is dynamic and any change due to references updating can be managed through different approaches:
    1. The attribute can be kept inconsistent in the db until the next reading when

it will be recalculated. This is the worst case scenario because the number of reading is much higher than the writing one,therefore this behavior is inefficient;

2. The attribute can be kept inconsistent in the db until it will be recalculated by a trigger after a certain amount of time. This context is not entirely problem-free because there are small chances that the trigger doesn't update the value in time.

3. The attribute is always kept consistent after an applied operation. The score is update after three precise operation about the publication: create,delete and update. All this is fine because the writing operation frequency is lower than the reading one.

2 | ER Diagram

2.1. Entities and Relationships

The designed ER diagram contains the following entities: Person generic, Researcher, Book, Article, Journal, Proceeding, Inproceeding, Incollection, Publisher, Thesis, Online Resource, Association, Serie. Some classes have been introduced due to ER diagram correctness reasons, but they haven't been taken into account during the design of the Graph Database for ease of management of the DB as delign in 3.2. Entities are retrieved through a process of data analysis in which the hierarchy has been structured according to the XML file, instead, attributes are selected in a way that only the most relevant ones are kept for the project.

2.2. ER Diagram description

The Person Generic Entity is a Superclass identified by the CF and its specialized subclass is Researcher with a unique attribute Orcid which provides a persistent digital identifier distinguishing every researcher, this choice has been made to differentiate an amateur that doesn't make any formal publication from a professional researcher. The generalization is disjoint and partial. Every publication such as Book, Article, Thesis, Online Resource, Proceeding, Inproceeding, Incollection is made unique by its key and its publication date. Proceeding is a collection of academic papers published in the context of an academic conference or workshop which contains the contributions made by researchers(Inpreceeding) at the conference. The Entity Incollection is referred to the entries that are written from an author that is different from the one that has drawn up the book. In the Er-Diagram we have the Entity Journal that represent a particular Scientific Magazine, and it's different from the Entity Volume that represent a particular edition of the journal (it contains its own Date of publication and has a number referred to the Journal). It is important to consider that thesis are written in University, that is an entity. Series are collections of publications which refer to a common topic or theme, they are identified by their title. These are the attributes contained by each relationship:

- Book_in between Book and Incollection: Page end and Page start;
- incollection_Series: Number;
- Book_Series: Number;
- Preceeding_Series: Number;
- Article_Number: Page end and Page start.

The ER model is below: (note that arrows in the relationships are not needed in the ER)

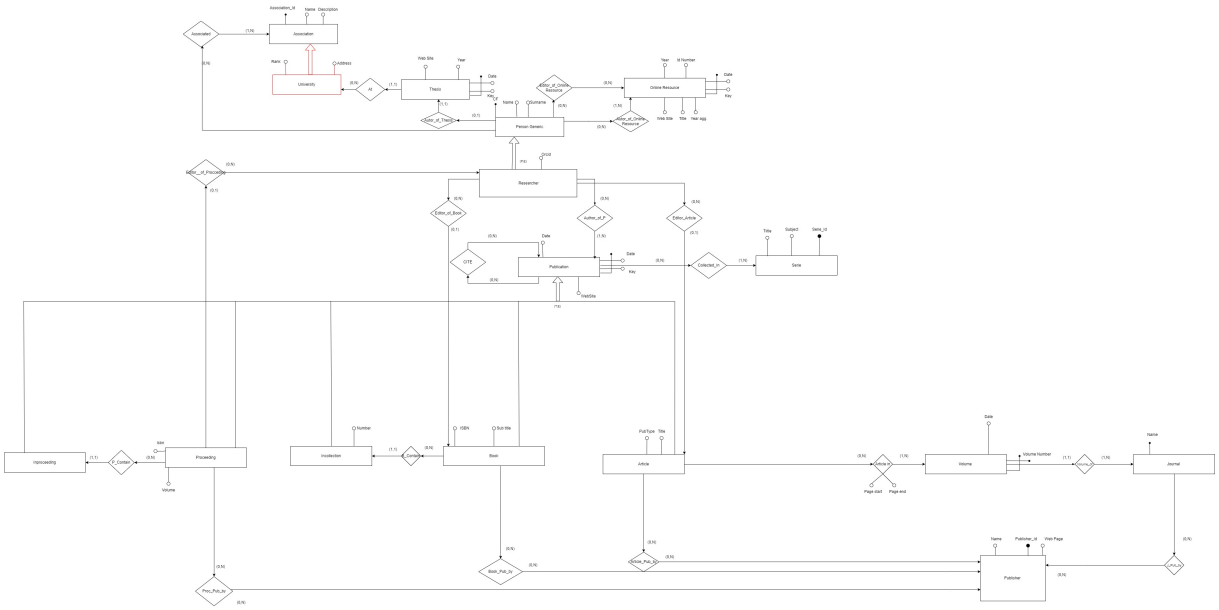


Figure 2.1: ER Diagram

3 | Graph Diagram

3.1. Graph Diagram description

Author, Article, Journal, Book, Incollection, Proceedings, Inproceedings, Www, Mastersthesis, Publication, University and Serie are Node labels and their attributes are shown in the graph diagram below. The relationships of the database are WRITTEN_BY, CONTAIN, PUBLICATED_BY, COAUTHOR, AT, CITE, INCLUDE. We omitted some links using the publication note make graph diagram more readable and understandable.The nodes Article, Book, Incollection, Proceedings, Inproceedings, Www, Mastersthesis contain the Publication tag in order to perform and understand queries easily since you don't need to specify 7 different nodes in the MATCH statement every time. The relationships between nodes are the following:

```
(Publication)-[WRITTEN_BY]->(Author)
(Proceedings)-[CONTAIN]->(Inproceedings)
(Book)-[CONTAIN]->(Incollection)
(Mastersthesis)-[AT]->(University)
(Author)-[PAST_STUDENT]->(University)
(a1:Author)-[COAUTHOR]-(a2:Author)
(p1:Publication)-[CITE]->(p2:Publication)
(Article)-[PUBLICATED_BY]->(Journal)
(Serie)-[INCLUDE]->(Publication)
```

Publication and Author nodes have the citationScore attribute. In a particular for publication it is used to store a computed index which describes how many times that node is quoted. This attribute is updated by using a specific command that executes the PageRank algorithm. The citationScore attribute will be used in queries to classify authors and publications. On the other hand, the CitationScore referred to an Author is computed considering the number of the total citations received and the average amount of citation for each publication. The relationship COAUTHOR is introduced to increase the speed of the queries searching for the relationships between authors. Two authors are connected

by the coauthor relationship only if they have participated to the writing of the same publication. The implementation of the relationship PAST_STUDENT was made to optimize the queries on the MasterThesis publication type. Its purpose is to associate an author who wrote a thesis directly with its university. If an author has not published a thesis Past_Student Relationship represent the university associated with. Journals are represented as nodes and described only by their name, it has a relationship with Article sharing the publication details (year, month, volume, pages, number). By doing this it is possible to efficiently aggregate all the articles published in the same journal without creating one node for each number of that journal. The database design decision follows two thoughts:

1. be fast, the core of the structure is able to easily guarantee the functioning of complex algorithm like shortest path, thanks to the quantity of relations;
2. be used for ranking authors and articles based on citations(for example we are refering to citationScore).

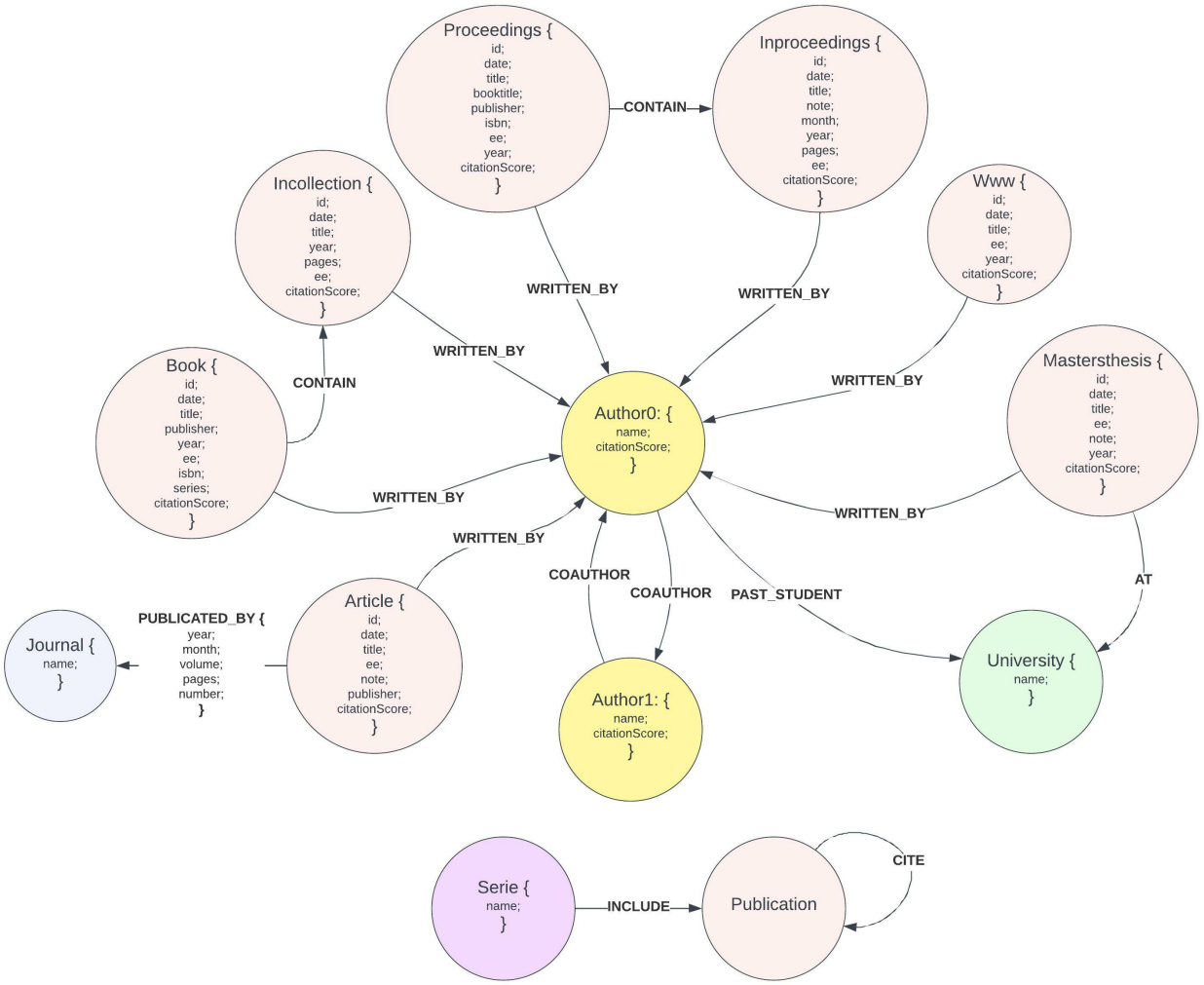


Figure 3.1: Graph Diagram

3.2. ER Diagram and Neo4j differences

The complete and general ER model that has been created represents an example of a real scientific bibliography. As requested in neo4j we worked in order to get the best from the tool used and the type of database. There are some differences in the treatment of the authors in neo4j (simple nodes with some attributes) and the authors in the ER diagram (researchers + generalization in a generic person) to differentiate the ones with ORCID and the ones without, since in reality only those who own the ORCID can write a book or scientific article, while students or other generic people can still write online resources (Www) or write a thesis (Mastersthesis).Journal entity is more detailed in the ER diagram (with weak entities such as Volume and Number reflecting several more specific releases, and an entity "Publisher" that publish a book or a proceeding on the journal) then the

graph diagram with some attributes in the relationship “Publicated_by”. The University node is an entity in the ER diagram. We have several authors in the graph diagram to have a "co-author" relationship that simply are authors of the same publications (or others). Finally, the difference in the treatment of the series entity which, thanks to the parent entity "Publication", is treated in a more orderly and concise way in the ER diagram compared to neo4j (due to the absence of true generalizations).

4 | Dataset Description and processing

The chosen Dataset is the DBLP computer science bibliography. It is the on-line reference for bibliographic information on major computer science publications. As January 2019 DBLP contains over 4.4 million publications, published by more than 2.2 million authors. The Dataset is divided in different types of publication (article, book, incollection, proceedings, inproceedings, www, mastersthesis) followed by their features and details. The attributes and relationships of these types are already described in the ER Diagram section 2. The Dataset has been subject to a process of elaboration and extraction in order to clear the data and take care of their quality, indeed, limited quality data could "cause a series of consequences for efficiency". The starting point was cut the 3.5 Gigabytes of data into 100 Megabytes (more or less 200.000 nodes) and convert its XML format into CSV. These operations were made much easier through the use of two Python programs:

1. The first one 4.1 aims at converting the entity names or entity numbers of the HTML characters with its original character representation in the encoded UTF-8 string and at cleaning up the non-standard xml code. It is important to highlight that the program realizes the reading of the first 100 Megabytes of the Dataset performing a data cut.

```python
import html
import fileinput

totMB = 100

inputFile="D:/Uni/Magistrale - 1/SMBUD/Progetto/Data/dblp.xml"
outputFile="D:/Uni/Magistrale - 1/SMBUD/Progetto/Data/out"+str(totMB)+"M.xml"

input = open(inputFile,'rb')
text = input.read(totMB*1000000)
output = open("temp.xml",'wb')
output.write(text)
output.close()

output = open(outputFile, 'a', encoding="UTF-8")
output.truncate(0)
count = 0

for lines in fileinput.input(["temp.xml"]):
    line = lines.replace("<i>", "")
    line = line.replace("</i>", "")
    line = line.replace("<sup>", "")
    line = line.replace("</sup>", "")
    line = line.replace("<sub>", "")
    line = line.replace("</sub>", "")
    line = line.replace("&lt;", "")
    line = line.replace("&gt;", "")
    line = line.replace('&amp;', " and ")
    uLine = html.unescape(line)
    output.write(uLine)
    count+=1
    if count % 1000000 == 0:
        print(count)


output.close()
```

Figure 4.1: First Python program

2. The second one 4.2 carries out the conversion from XML to CSV and removes the special characters from CSV. One more thing to mention is that after all types of publications are found only a limited number is stored for each type (e.g. a maximum number of the type book and of the type article is saved fixing an upper bound at 2000).

```python
import xml.etree.ElementTree as ET
import csv
import math
import random
import copy

input_file = "out100M.xml"
num_limit = 2000

tree = ET.parse(input_file)
root = tree.getroot()

csvIndex = {}
csvHeaders = {}
keyListProceedings = []
keyListBooks = []

#find all types of pubblications and loads to a dictionary
for child in root:
    if child.tag not in csvIndex:
        csvIndex[child.tag]={}
        csvHeaders[child.tag]=[]

#scan all pub types
#maxDict object to store the maximum number of attributes for each pubblication type
for pub_type in csvIndex.keys():
    maxDict = {}
    currIndex = 2
    counter = 0

    for elem in root.findall(pub_type):
        thisDict = {}

        if pub_type == "proceedings":
            keyListProceedings.append(elem.attrib["key"])
            #save the key of that article in a list for randomic assigment

        if pub_type == "book":
            keyListBooks.append(elem.attrib["key"])
            #save the key of that article in a list for randomic assigment

        for innerTag in elem:
            #counts the number of each tags for an element
            if innerTag.tag in thisDict:
                thisDict[innerTag.tag]+=1
            else:
                thisDict[innerTag.tag]=1
```

```
48                #we create the tag in the dictionary
49
50            #updates the total count for that pubblication type
51            if innerTag.tag in maxDict:
52                if thisDict[innerTag.tag] > maxDict[innerTag.tag]:
53                    maxDict[innerTag.tag] = thisDict[innerTag.tag]
54            else:
55                maxDict[innerTag.tag]=1
56                #we create the tag in the dictionary
57
58        counter += 1
59        if counter>=num_limit:
60            break
61
62    #load the found number of attributes for
63    #that pubblication type in the global model of indexes
64    for key in maxDict.keys():
65        csvIndex[pub_type][key] = currIndex
66        currIndex = currIndex+maxDict[key]
67
68        csvHeaders[pub_type] = ["date","key"]
69        for key in maxDict.keys():
70            #creates the header for the csv
71            for n in range(maxDict[key]):
72                csvHeaders[pub_type].append(key+str(n))
73
74
75
76 #loads the data in the csv one type at time
77 for pub_type in csvIndex.keys():
78     counter = 0
79     out_csv = open("./csv/"+pub_type+'.csv','w', newline='', encoding='UTF-8')
80     csvwriter = csv.writer(out_csv)
81
82     csvwriter.writerow(csvHeaders[pub_type])
83     #write the header row
84
85     for elem in root.findall(pub_type):
86         row = [""] * len(csvHeaders[pub_type]) #array initialization
87         tmpIndex = copy.deepcopy(csvIndex)
88         #duplicate the index model for writing
89
90         #load fixed attributes of pubblications
91         row[0] = elem.attrib["mdate"]
92         row[1] = elem.attrib["key"]
93
94         #load dynamic values of tags
```

```
95          for innerTag in elem:
96              row_pos = tmpIndex[pub_type][innerTag.tag] #get the write column position
97              if innerTag.tag == "crossref" and pub_type == "incollection":
98                  row[row_pos] = random.choice(keyListBooks)
99              elif innerTag.tag == "crossref" and pub_type == "inproceedings":
100                 row[row_pos] = random.choice(keyListProceedings)
101             else:
102                 row[row_pos] = innerTag.text
103                 #write the corrisponding value
104
105             tmpIndex[pub_type][innerTag.tag]+=1
106             #increment the writing position
107
108         csvwriter.writerow(row)
109         counter += 1
110         if counter>=num_limit:
111             break
112
113     out_csv.close()
```

Figure 4.2: Second Python program

The final CSV result of this process is further analyzed and elaborated by deciding what to include or not in the Neo4j DB based on quantity of data present (erasing the weak and irrelevant data and relations). After doing this, the relationship between the field Book_Incollection and Proceeding_Inproceeding are generated randomly because the Database is not considered in its entirety, indeed, it is not certain that the original correlations have been remained unaltered or have been lost during the cutting operation. The last step has been the DB upload in Neo4j through the Cypher queries in particular using the command Load as we will see in section 5.19 .

5 | Queries and Commands

5.1. Query 1

This query finds for the specified journal X the authors that have written for it and the number of their respective publications in a certain period of time. The matched results are sorted in descendent through the ORDER BY clause and the query returns a limited subset of the rows through the LIMIT clause. In this example the query returns the top 10 author of IWBS Report during the period between 1990 to 2000.
Complexity 3 nodes + conditions + aggregations + limit

```
WITH '1990' AS startYear, '2000' AS endYear, 'IWBS Report' AS journalName
MATCH (aut:Author)<-[:WRITTEN_BY]-(a:Article)-[r:PUBLICATED_BY] ->(j:Journal)
WHERE j.name = journalName AND r.year >= startYear AND r.year <= endYear
WITH aut, j, COUNT(*) AS numArtXAut
ORDER BY numArtXAut DESC
RETURN aut.name, numArtXAut
LIMIT 10
```

	Name of the author	Number of articles x author
1	Christoph Beierle	8
2	Egon Börger	7
3	Stefan Böttcher	6
4	Ulrike Schwall	5
5	Stefan J. Schierholz	4
6	Udo Pletat	4
7	Gregor Meyer	3
8	Jochen Dörre	3
9	Ulrike Weiland	3
10	Claudia Maienborn	3

5.2. Query 2

This description of this query has been improved!

This query consider, for a certain Author ('Tor Helleseth'), all the incollections written in different books. For each book the query compute the contribution percentage of the author obtained as the ratio between the number of incollections written in that book (also from other authors) and the total number of incollections.

Complexity 3 nodes + conditions + aggregations

```
MATCH (inc:Incollection)-[:WRITTEN_BY]->(aut:Author),(b:Book)-[:CONTAIN]->(inc)
WHERE aut.name = 'Tor Helleseth'
WITH b, COUNT(*) AS numIncXBook
MATCH (b)-[:CONTAIN]-(inc:Incollection)
WITH b, numIncXBook, COUNT(*) AS numTotIncXBook
RETURN b.title,
CASE
WHEN numIncXBook = 0 THEN 0.0
WHEN numTotIncXBook = 0 THEN 0.0
ELSE (numIncXBook * 1.0 / numTotIncXBook * 1.0) * 100.0 END AS Percentage
```

	Book title	Rating
1	Handbook on Graph Drawing and Visualization	5.26315
2	Recommender Systems Handbook	4.16667
3	Encyclopedia of Complexity and Systems Science	11.76471
4	Handbook of Discrete and Computational Geometry, Second Edition	12.5
5	"Encyclopedia of Parallel Computing"	5.26315
6	Encyclopedia of Data Warehousing and Mining (4 Volumes) Second Edition (4 Volumes)	4.54546
7	Information security management handbook, Sixth Edition	5.55556
8	Encyclopedia of Computational Neuroscience	5.0
9	Handbook of Algorithms for Physical Design Automation	4.34782
10	Handbook of Formal Languages, Volume 2. Linear Modeling: Background and Application	3.84615
11	Handbook of Information and Communication Security	4.54546
12	Handbook of Semantic Web Technologies	6.25
13	Handbook of Mathematical Methods in Imaging	6.66667
14	Handbook of Virtual Environments - Design, Implementation, and Applications, Second Edition	5.0
15	Handbook of Spatial Logics	5.26315
16	Handbook of Database Security - Applications and Trends	4.34782
17	The Handbook of Mobile Middleware	6.66667

5.3. Query 3

This query returns the articles of journal with their respective author that are the published in a certain period of time. The result is sorted in descendent by the publication date and it is a limited subset of the rows through the limit clause. In this case the query returns in descendent order the 10 article of journal with their respective author during the period between 1980 to 1990.
Complexity 3 nodes + conditions + aggregation + limit

> We are aware that the complexity condition does not meet the description. So it should be excluded by the formal 10 queries.

```
MATCH (aut:Author)<-[:WRITTEN_BY]-(a:Article)-[r:PUBLICATED_BY]->(j:Journal)
WHERE r.year>='1980' and r.year<= '1990'
WITH a.title AS articleTitle, r.year AS pubYear,aut.name as Authors
```

```
ORDER BY pubYear DESC
RETURN articleTitle, authors, pubYear
LIMIT 5
```

	articleTitle	author	pubblication year
1	Object Data Model Facilities for Multimedia Data Types	Mark F. Hornick Frank Manola Alejandro P. Buchmann	1990
2	Object Oriented DBMS as Generalization of Relational DBMS	David Beech Cetin Ozbutun	1990
3	Modification of User Defined Types	Phil Shaw	1990
4	Modelling Spatial Knowledge on a Linguistic Basis	Geoffrey Simmons Ewald Lang Kai-Uwe Carstensen	1990
5	Das Problem der VP-Ellipsen im Rahmen einer Theorie der Sachverhaltsanaphern	Elke Busch	1990

5.4. Query 4

This query returns the title of 10 works published by a specific journal over the last year with the authors who wrote it. The max() function returns the maximum value in a set of values and after the matching condition are applied. In this case the query returns the works published in the last year by the journal IWBSReport and their authors
Complexity 3 nodes + conditions + aggregations + limit

```
MATCH(a:Author) <- [:WRITTEN_BY] - (p:Publication) -[r:PUBLICATED_BY] - (j:Journal)
WITH j,r,p,a,max(r.year) AS maxYear
WHERE j.name = 'IWBS Report' AND r.year = maxYear
RETURN p.title,collect(a.name) AS authors
LIMIT 10
```

	p.title	authors
1	ILFA Interface Concept	["Wolfgang Schönfeld", "Theodor Lettmann", "Bernd I. Dahn"]
2	Modelling Spatial Knowledge on a Linguistic Basis	["Geoffrey Simmons", "Ewald Lang", "Kai-Uwe Carstensen"]
3	Domain Restriction and Other Kinds of Reference Set Operations in Sentence and Discourse Semantics.	["Rolf Mayer"]
4	Das Problem der VP-Ellipsen im Rahmen einer Theorie der Sachverhaltsanaphern	["Elke Busch"] ["Elke Busch"]
5	Syntaktische Informationseinheiten zu deutschen Adjektiven	["Stefan J. Schierholz"]
6	Complexity of Logical Decision Problems: An Introduction	["Egon Börger"]
7	Position und Bewegung: Zur Semantik lokaler Verben	["Claudia Maienborn"]
8	On Subsumption and Semiunifaction in Feature Algebras	["William C. Rounds", "Jochen Dörre"]
9	Derivationsmorphologie in einem Übersetzungssystem	["Anne Schiller"]
10	Knowledge Representation in LILOG	["Kai von Luck", "Udo Pletat"]

5.5. Query 5

This query returns a ranking of the first 10 authors that wrote on a specific journal. The score of each author is calculated using the average citation score of its publications adding a weight based on the number of publications. After matching all publications of an author for that journal, loading them as a collection using the collect() function and elaborating them, the results are ordered based on the FinalScore value and limited to 10 using the LIMIT function.
Complexity 3 nodes + conditions + aggregations + limit

```
MATCH (a:Author)<-[:WRITTEN_BY]-(p:Publication)-[:PUBLICATED_BY]->(j:Journal)
WHERE j.name='Sci. Eng. Ethics'
WITH collect(p) AS pubs, sum(p.citationScore) AS
totScore,a, size(collect(p)) AS numPubs
RETURN a.name as Name, totScore, numPubs, totScore/numPubs AS
meanScore, 0.1*totScore+0.9*(totScore/numPubs) AS FinalScore
```

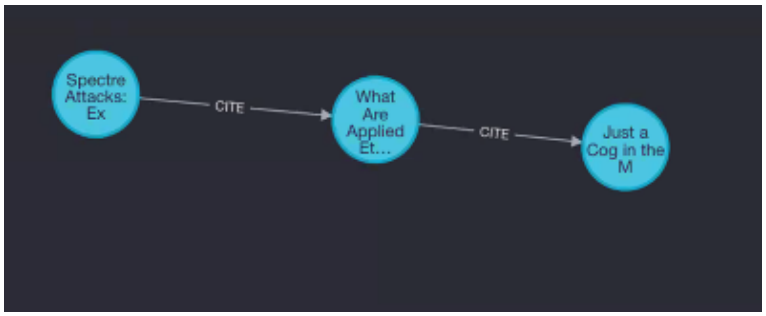
```
ORDER BY FinalScore DESC
LIMIT 10
```

	Name	totScore	numPubs	meanScore	FinalScore
1	Michael D. Mumford	16.00781	16	1.00048	2.50122
2	Stephanie J. Bird	14.94547	15	0.99636	2.39127
3	Luciano Floridi	15.08879	16	0.94304	2.35762
4	Jaime A. Teixeira da Silva	13.24581	14	0.94612	2.17609
5	Neelke Doorn	13.00399	14	0.92885	2.13637
6	David B. Resnik	12.07406	12	1.00617	2.11296
7	Tanuj Kanchan	11.86092	12	0.98841	2.07566
8	Ana Marusic	11.64288	12	0.97024	2.03750
9	Shane Connelly	11.63754	12	0.96979	2.03657
10	Kewal Krishan	10.98351	11	0.99850	1.99700

5.6. Query 6

This query uses the neo4j function shortestPath() to find the shortest path between two nodes using CITE relations. In this case the query search for publications "Just a Cog in the Machine? The Individual Responsibility of Researchers in Nanotechnology is a Duty to Collectivize." and "Spectre Attacks: Exploiting Speculative Execution.". To have a relevant result we impose a minimum of 2 nodes in the shortest path.

```
MATCH (p1:Publication{title:"Just a Cog in the Machine?
The Individual Responsibility of Researchers in Nanotechnology
is a Duty to Collectivize."}),
(p2:Publication{title:"Spectre Attacks: Exploiting Speculative Execution."}),
sp= shortestPath((p1)-[:CITE*]-(p2))
WHERE length(sp)>1
RETURN sp
```



5.7. Query 7

This query returns a ranking of the first 10 authors that wrote on a journal in a given year. The score of each author is calculated using the average citation score of its publication adding a weight based on the number of publications. The result is ordered using the FinalScore of each author. In this case the used year for the query is 1997.

Complexity 3 nodes + condition + aggregations + limit

```
MATCH (a:Author)<-[:WRITTEN_BY]-(p:Publication)-[r:PUBLICATED_BY]->(j:Journal)
WHERE r.year = '1997'
WITH collect(p) AS pubs, sum(p.citationScore) AS
totScore,a, size(collect(p)) AS numPubs
RETURN a.name AS Name, totScore, numPubs, totScore/numPubs AS
meanScore, 0.1*totScore+0.9*(totScore/numPubs) AS FinalScore
ORDER BY FinalScore DESC
LIMIT 10
```

	Name	totScore	numPubs	meanScore	FinalScore
1	Victor Klee	1.35527	1	1.35527	1.35527
2	Peter Gritzmann	1.35527	1	1.35527	1.35527
3	Jochen Bern	1.30958	1	1.30958	1.30958
4	Christoph Meinel	4.67460	5	0.93492	1.30888
5	Helmut Seidl	2.92863	3	0.97621	1.17145
6	Oleg V. Khamisov	1.15070	1	1.15070	1.15070
7	Christian Fecht	2.02935	2	1.01467	1.11614
8	Reiner Horst	1.11138	1	1.11138	1.11138
9	Le Dung Muu	1.11138	1	1.11138	1.11138
10	Belinda B. King	1.07856	1	1.07856	1.07856

5.8. Query 8

This query returns the ranking of the first 4 universities ordered by the number of students who have completed a thesis at it from the year 2000. The number of thesis is found using the size() function. The result is ordered according to the number of past students at that university. NumOfPublic is the number of thesis.
Complexity 3 nodes + conditions + aggregations + limit

```
MATCH (p:Author)<-[:WRITTEN_BY]-(t:Mastersthesis)-[:AT]->(u:University)
WHERE t.date> '2000-01-01'
WITH collect(t.title) AS Thesis, u, size(collect(t.title)) AS
numPublication, collect(p.name) AS
pastStudents,size(collect(p.name)) AS numPastStud
RETURN u.name AS University, Thesis, numPublication AS NumOfThesis,
pastStudents, numPastStud
ORDER BY numPastStud DESC
LIMIT 4
```

	University	Thesis	NumOfThesis	PastStudents	NumPastStud
1	Cambridge	["Portierung des DBLP-Systems auf ein relationales Datenbanksystem und Evaluation der Performance.", Who's Who in the World Wide Web: Approaches to Name Disambiguation", "Der Einfluss kleiner naturnaher Retentionsmaßnahmen in der Fläche auf den Hochwasserabfluss -Kleinrückhaltebecken -."]	3	["Stephan Vollmer", "Vanessa, C. Klaas", "Rita Ley"]	3
2	Stanford	["PRPL: A Database Workload Specification Language, v1.3."]	1	["Kurt P. Brown"]	1
3	Harvard	["Efficient View Maintenance at Data Warehouses."]	1	["Tolga Yurek"]	1
4	Oxord	["Regelbasierte Extraktion und asymmetrische Fusion bibliographischer Informationen."]	1	["Oliver Hoffmann 0002"]	1

5.9. Query 9

This description of this query has been improved!

This query take from the "MIT" University the best 10 students for the number of times that journals contained their publications, considering only publications after 2000.
Complexity 4 nodes + conditions + aggregations + limit

```
MATCH(u:University)<-[:PAST_STUDENT]-(s:Author)<-[:WRITTEN_BY]-(p:Publication)-
[:PUBLICATED_BY]->(j:Journal)
```

```
WHERE p.date>="2000-01-01" AND u.name="MIT"
WITH s.name AS student, collect(DISTINCT(j.name))
AS journals, size(collect(j.name)) AS numJournal
RETURN student, journals, numJournal
ORDERBY numJournal DESC
LIMIT 10
```

	student	journals	numJournal
1	Dieter Baum	["Universität Trier, Mathematik/Informatik, Forschungsbericht"]	21
2	Stephanie J. Bird	["Sci. Eng. Ethics"]	15
3	Tanuj Kanchan	["Sci. Eng. Ethics"]	12
4	Stefan Böttcher	["IWBS Report"]	9
5	Michael W. Kalichman	["Sci. Eng. Ethics"]	8
6	Bert Gordijn	["Sci. Eng. Ethics"]	8
7	Aceil Al-Khatib	["Sci. Eng. Ethics"]	7
8	Roland Seiffert	["IWBS Report", "LILOG-Memo", "LILOG-Report"]	6
9	Jochen Bern	["Universität Trier, Mathematik/Informatik, Forschungsbericht"]	6
10	Patricia Osseweijer	["Sci. Eng. Ethics"]	6

5.10. Query 10

In this query we search the ranking of Universities by number of citations in 2019, so we have to search for the relations with every author that was a student in the specific University and count the number of publications.
Complexity 3 nodes + conditions + aggregations + limit

```
MATCH (u:University)<-[:PAST_STUDENT]-
(a:Author)<-[:WRITTEN_BY]-(p1:Publication)<-[c:CITE]-(p2:Publication)
WHERE '2019-01-01'<=p1.date <= '2020-01-01'
WITH u.name AS University, size(collect(p2)) AS NumPublications
RETURN University, NumPublications
ORDER BY NumPublications DESC
LIMIT 5
```

	University	NumPublications
1	Toronto University	3219
2	Cambridge	1515
3	MIT	1470
4	Oxord	1462
5	Polimi	1058

5.11. Query 11

This query classifies the Universities that have the highest number of students that have published in the economic field (to do this it is used the Serie node), limited to five.
Complexity 3 nodes + conditions + aggregations + limit

```
MATCH(s:Serie)-[:CONTAIN]->(p:Publication)-[:WRITTEN_BY]->(a:Author)-[past:PAST_STU
WHERE s.name="AllAboutEconomic"
```

A really rare corner case was not controlled.

```
WITH collect(distinct (a.name)) as Students , u
WITH Students, size(Students) as Num_Students, u

RETURN Students, Num_Students, u.name
ORDER BY Num_Students DESC
LIMIT 5
```

	Students	NumStudents	University
1	["Jong Yong Abdiel Foo", "Marc Vauclair", "Brian Bowe", "Tim Berners-Lee", "Jong Yong Abdiel Foo", "Dirk Schnelle"]	6	Cambridge
2	["Olivier Ridoux", "Anne Canteaut", "Diana Adela Martin", "Roy T. Fielding", "Paul C. Leyland"]	5	Oxord
3	["Tor Helleseth", "Rosalie J. Ocker", "Eddie Conlon"]	3	Stanford
4	["Henrik Frystyk Nielsen", "Helmut Seidl"]	2	Harvard
5	["Friedrich L. Bauer"]	1	MIT

5.12. Query 12

In this query there is a match of series that contains a type of publications with the quotes to other publications. After there is (again) a match with all the quotes to publications (that are in the specific serie, except other that are already in that serie). Finally there

is a collect of the number of citations (found before) in a best 5 classification.
Complexity 3 nodes + conditions + aggregations + limit

This is a better version of the past query

```
MATCH(s:Serie)-[:CONTAIN]->(p1:Publication)<-[c:CITE]-(p2:Publication)
WITH s.name as SerieName, count(c) as Num_Citations
RETURN  SerieName, Num_Citations
ORDER BY Num_Citations DESC
LIMIT 5
```

	SeriesName	NumCitations
1	AllAboutEconomic	924
2	AllABoutComputerScience	642
3	AllAboutAi	471
4	AllAboutSocialScience	316
5	AllAboutPhysic	188

5.13. Query 13

This query classify the 5 most important authors by the number of coauthors they have.
To work it uses the following functions from gds and apoc libraries:

- Gds.graph.drop (to drop a graph based on the data input)
- Gds.graph.project (to create a sort of view without any directed link in this case)
- Gds.degree.stream (to stream an output data from the graph)
- Apoc.when

```
CALL gds.graph.exists('coauthorsUn')
YIELD exists AS ex
CALL apoc.when(ex, 'CALL gds.graph.drop("coauthorsUn") YIELD graphName AS G' )
YIELD value AS val
CALL gds.graph.project( 'coauthorsUn', 'Author', {
COAUTHOR: { orientation: 'UNDIRECTED' }})
YIELD graphName AS coauthorsGraph
CALL gds.degree.stream(coauthorsGraph)
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS authorName, score AS numbeOfCoauthor
ORDER BY numbeOfCoauthor DESCENDING, authorName
```

```
LIMIT 5
```

	AuthorName	NumberOfCoauthor
1	Ana Marusic	48.0
2	Michael D. Mumford	36.0
3	Ivan Buljan	35.0
4	Ellen-Marie Forsberg	29.0
5	Hub A. E. Zwart	26.0

5.14. Query 14

For a certain university named "MIT", this query returns the ranking of the 3 most famous proceedings counting the total number of citations (proceeding + inproceedings) from 2000 onwards.
Complexity 5 nodes + conditions + aggregations + limit

```
MATCH (u:University)<-[:PAST_STUDENT]-(aut:Author)<-[:WRITTEN_BY]-
(p:Proceedings)-[:CONTAIN]->(i:Inproceedings)<-[:CITE]-(pub:Publication)
WHERE u.name="MIT" AND pub.date >="2000-01-01"
WITH size(collect(pub)) AS NumCitation , p, aut
MATCH (p)<-[:CITE]-(pub2:Publication)
WITH aut, p, NumCitation+size(collect(pub2)) AS TotNumCitation
RETURN aut.name, p.title, TotNumCitation
ORDER BY TotNumCitation DESC
LIMIT 3
```

	Author	Proceeding	TotNumCitation
1	Joaquim Filipe	Transactions on Computational Collective Intelligence XVII	283
2	Joaquim Filipe	Transactions on Computational Collective Intelligence XX	195
3	Stéphane Loiseau	Transactions on Computational Collective Intelligence XX	195

5.15. Command 1

This command is used to update the citationScore attribute of the publications. In the first part of the command it is checked if already exist a graph called "citationsGraph" in the gds library. If this graph already exist, it is dropped. In the second part of

the command we create a new graph in the gds library (using the command project) containing all the CITATION relationships between publications. Finally on this graph it is launched the PageRank algorithm (gds.pageRank.stream) which calculates all the indexes that will be written as attribute in the corresponding publication nodes.

```
CALL gds.graph.exists('citationsGraph')
YIELD exists as ex
CALL apoc.when(ex, 'CALL gds.graph.drop("citationsGraph") YIELD graphName AS G')
YIELD value AS val
CALL gds.graph.project.cypher('citationsGraph',
'MATCH (n :Publication) WHERE n.consistencyStatus=false
RETURN id(n) AS id',
'MATCH (n)-[c:CITE]->(p) RETURN id(n) as source, id(p) as target ,
type(c) as type, c.weight as weight')
YIELD graphName as graph, nodeCount, relationshipCount
CALL gds.pageRank.stream(graph)
YIELD nodeId, score
MATCH(n:Publication)
WHERE id(n) = nodeId
SET n.citationScore =score
SET n.consistencyStatus=true
```

5.16. Command 2

This command updates all the citationScore of the authors who have consistencyStatus = False. It exploits an algorithm already used in the previous queries (ex. Query 5-7).

```
MATCH (a:Author)<-[:WRITTEN_BY]-(p:Publication)
WHERE (a.consistencyStatus) = False
WITH collect(p) AS pubs, sum(p.citationScore)
AS totScore,a, size(collect(p)) AS numPubs
WITH a, totScore, numPubs, totScore/numPubs AS
meanScore, 0.1*totScore+0.9*(totScore/numPubs) AS FinalScore
SET a.citationScore = FinalScore
SET a.consistencyStatus= TRUE
```

5.17. Command 3

This command deletes all proceeding and inproceeding linked to a certain author. In this case the name of the author is: “Dieter Baum”. In order to maintain the database cosistent we need to modify the CitationScore related to the Publication Cited by the Proceeding and the Inproceedings to remove. We use a Boolean Attribute, ConsistencyStatus that is set to false if we need to recompute the Citation score related to the same node. The query for this reason updates the consistency status of the related Nodes. We choose to update the correct CitationScore attribute with a trigger that can be arised strategically considering traffic peaks and other external conditions.

```
MATCH (aut:Author)<-[:WRITTEN_BY]-(proceeding:Proceedings)-[:CITE]->
(p:Publication)-[:WRITTEN_BY]->(otherAut:Author)
WHERE aut.name ="Dieter Baum"
WITH proceeding , p, otherAut, aut
SET aut.consistencyStatus=FALSE
SET otherAut.consistencyStatus=FALSE
SET p.consistencyStatus=FALSE
WITH proceeding
OPTIONAL MATCH (procceding1 :Proceedings)-[:CONTAIN]->
(inproceedings:Inproceedings) -[:CITE]->(p2:Publication)-
[:WRITTEN_BY]-> (otherAut2 :Author)
WITH proceeding, inproceedings , p2, otherAut2
SET otherAut2.consistencyStatus=FALSE
SET p2.consistencyStatus=FALSE
DETACH DELETE proceeding, inproceedings
```

5.18. Command 4

This command creates an article published in a journal and that cite other publications of a certain author. In this case the article created is WRITTEN_BY “Paul Kocher”, PUBLICATED_BY “meltdownattack.com” and CITE "Spectre Attacks: Exploiting Speculative Execution." and "An Evaluation of Object-Oriented DBMS Developments: 1994 Edition."

```
WITH ["Spectre Attacks: Exploiting Speculative Execution.",
```

```
"An Evaluation of Object-Oriented DBMS Developments: 1994 Edition."]
AS citatedList
MATCH(p:Publication)
WHERE p.title IN citatedList
WITH p
MERGE(article:Article:Publication{citationScore:0.0,date:date(),ee:"https://=)",
id:" tr/marhanz/s00",title:"Happy Birthday Algoritm"})-[:WRITTEN_BY]->
(aut:Author{name:'Paul Kocher'})
MERGE (article)-[:PUBLICATED_BY {year:date().year}]->
(j:Journal{name:'meltdownattack.com'})
MERGE (article)-[:CITE]->(p)
```

5.19. Extra Commands

This section introduces some of the necessary operation used to structure the db by creating Nodes and their relationships. These operations are essentials to perform the previous queries.

5.19.1. Import in the Database

As mentioned in the previous chapter, the DB is imported in Neo4j from csv file through the cypher queries in particular using the command Load. Below are shown a couple of example where Www, Authors and Editor are upload in the database with their attributes. The Merge clause ensures that a pattern exists in the graph when the specified pattern is not present and needs to be created, any variables previously bound to existing graph elements will be reused in the pattern. With Header clause we are specifying that the file starts with a single row containing column names allowing specific fields access by their corresponding column name.

Load Www Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///www.csv' AS row
WITH row
MERGE (www:Www:Publication {id: row.key})
ON CREATE SET www.date = row['date'], www.title = row['title0'],
www.ee = row['ee0'], www.year = row['year0']
```

Load Authors of Www Nodes

```
LOAD CSV WITH HEADERS FROM 'file:///www.csv' AS row
WITH row
MATCH (www:Www {id: row.key})
UNWIND range(0,4) AS i
WITH i, www, row WHERE row['author'+toString(i)] IS NOT NULL
MERGE (aut:Author {name: row['author'+toString(i)]})
MERGE (www)-[:WRITTEN_BY]->(aut)
```

Load Www Editors as Authors

```
LOAD CSV WITH HEADERS FROM 'file:///www.csv' AS row
WITH row
MATCH (www:Www {id: row.key})
UNWIND range(0,5) AS i
WITH i, www, row WHERE row['editor'+toString(i)] IS NOT NULL
MERGE (aut:Author {name: row['editor'+toString(i)]})
MERGE (www)-[:WRITTEN_BY]->(aut)
```

5.19.2. Creation Coauthor relationship

This command creates a coauthor relationship between author of the same article and returns some data to analyze it. The list of author that has written the specified article is realized through the collect() function, then for each different authors the coauthor relationship is created, the query returns a limited subset of the rows through the limit clause.

```
MATCH (n:Article)-[w:WRITTEN_BY]->(a:Author)
WITH n,collect(a) AS authors
FOREACH( a1 IN authors |
FOREACH( a2 IN [a IN authors WHERE a<> a1| a]|
MERGE (a1)-[:COAUTHOR]-(a2)))
RETURN *
LIMIT 300
```

5.19.3. Creation Cite relationship

This command creates randomly Cite relationship between publication avoiding self-quotation. The range() returns a list comprising all integer values within a range bounded

by a start value start and end value, in this case an interval included between 1 and 120 as citationRange is considered. The apoc.coll.randomItem(collection) function returns a random item from the list and it is used to randomly choose the number of cite that each publication has and which are the cite they are linked to,then for each different publication the Cite relationship is created.

```
WITH range(1,120) AS citationRange
MATCH(p:Publication)
WITH collect(p) AS publications, citationRange
MATCH(p2:Publication)
WITH p2, apoc.coll.randomItems(publications,
apoc.coll.randomItem(citatioRange))  AS citatedPublications
FOREACH (citated IN [c IN citatedPublications WHERE c <>p2 | c]  |
CREATE (p2)-[:CITE]->(citated))
```

5.19.4. Create a relationship that associate the author to his past University

This command creates PASTSTUDENT relationship. The author is a PastStudent of a University if the Masterthesis was written by the author in the same University.

```
MATCH (p:Author)<-[:WRITTEN_BY]-(t:Mastersthesis)-[:AT]-(u:University)
MERGE(u)<-[:PAST_STUDENT]-(p)
```

Part II

Second Delivery: MongoDB

6 | Introduction

6.1. Specification

The aim of this second part is to design, structure and implement a document database in MongoDB to store a collection of articles.
Comparing this assignment to the first, the focus of the specification has moved from the generic description of the publications and their relationships to the detailed description of a single article.
In fact, it is asked to create a DB capable of containing many information about the article and also its complete text, divided into sections and paragraphs.

6.2. Hypothesis and assumptions

The database is implemented in this way:

- It was chosen to use only a database of articles, the other publication types were excluded because of the similarity of their structure;
- Imported data was randomly generated trying to keep the maximum likelihood to the real world data;
- The following database has been designed trying to maximize the efficiency of the queries, using whenever was possible sub-documents to directly access the data;
- In the database there may be authors who have not written articles yet;

7 | Document Description

7.1. Document schema

Article and *Author* are our collections of documents.

Each article contains:

- Relevant information stored in different attributes including the tile, an url representing the correlated online resource, date of publication.
- The doc_id that is a unique identifier associated with each article, it's assigned to each article when it's published.
- List of tags (such as #cloud)
- List of authors, stored as an array of *Author_preview* sub-documents. The *Author_preview* sub-document is designed with the aim to provide the most relevant information of an author that usually, in our application domain, it's accessed together with the related article.
- The *Text_data* sub-document which includes all the article text structure and the related data. It is designed with the aim to provide maximum flexibility in the article's structure. For this reason, the *Text_data* sub-document is made up of several *Section* sub-documents.
- The *Section* sub-document represent a section of an Article. It can be for example a chapter or a sub-chapter. In order to obtain this kind of structure, in which every chapter can include multiple sub-chapter, every Section includes an array of Section. Another advantage of this design choice is the flexibility it provides in terms of number of nesting-levels supported (potentially unlimited).
- Bibliography with the list of all the referenced articles, stored as an array of *Article_preview* sub-documents. An *Article_preview* sub-document include all the principal information related to an article that are usually accessed (such as title,

date of publication, doc_id) in every scenario where information of an article are presented in a compact way (such as list of article written by an author, bibliography).

Each author in the authors collection contains several attributes that give more personal information (such as biography, email, affiliation) and also a list of all the articles written by himself, stored as an array of *Article_preview*.

In the diagram sub-documents are represented using continuous line, while the references between different documents (by id) are represented using dashed line. The designed Document diagram is shown in figure 7.1.
An example of an article is shown in figure 7.2.

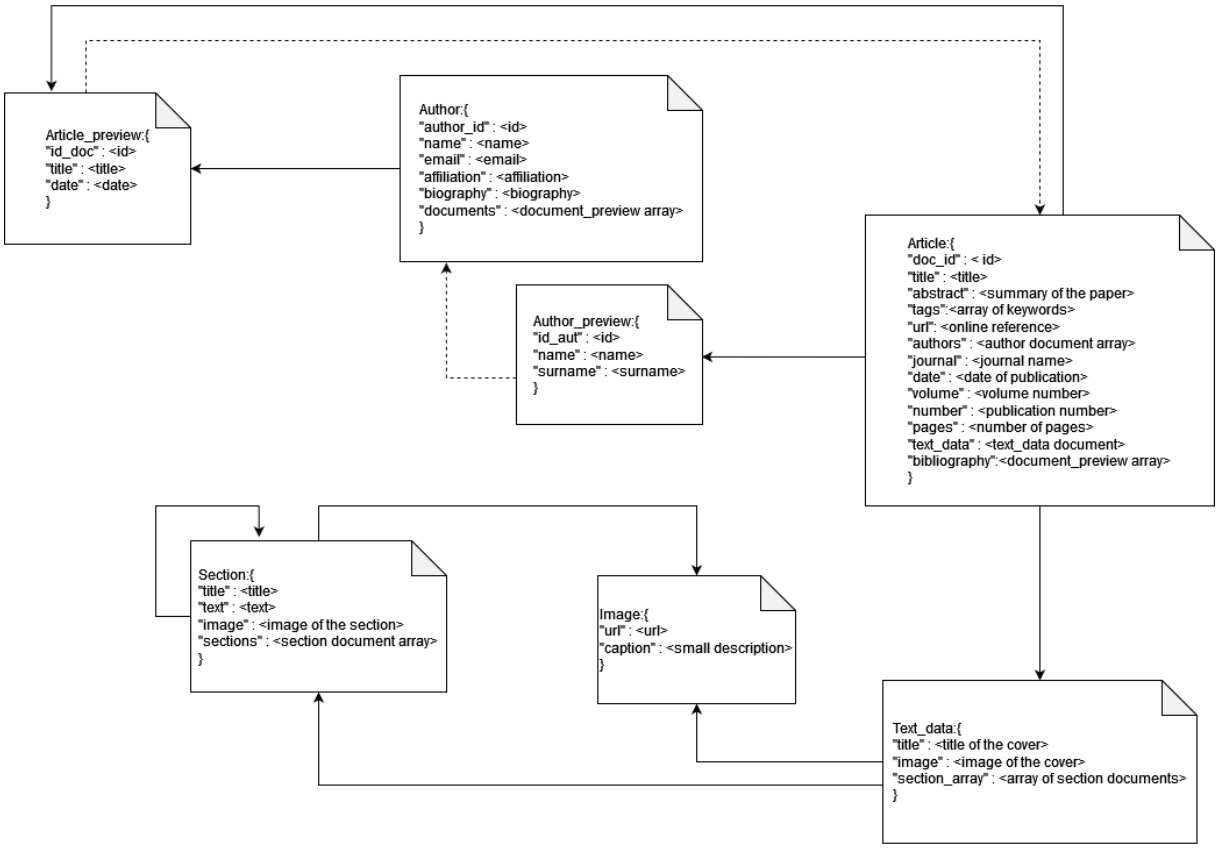


Figure 7.1: Document Diagram



Figure 7.2: Document Diagram

8 | Data Creation and Transformation

8.1. Pre-processing

The data was generated according to the document diagram using Mockaroo online tool and Python scripts. A portion of the DBLP database was uploaded as a dataset in Mockaroo and was used to randomly extract the author's name, the article title and the journal name.
The text content of the article was generated using a natural language dataset, which was cleaned up using a Python script and then imported in Mockaroo. All the other attributes were generated using the functions of the online tool. After that, the JSON files were elaborated using the second Python script shown below.

```
1   import csv
2   
3   out_csv = open('out_text.csv','w', newline='', encoding='UTF-8')
4   csvwriter = csv.writer(out_csv)
5   csvwriter.writerow(["id","text"])
6   
7   incremental = 0
8   
9   
10  for f_name in ["269-0.txt","282-0.txt","786-0.txt"]:
11      inputFile = open('./Sources/'+f_name, encoding='utf8')
12      lines = inputFile.read().split("\n\n")
13  
14      for i in range(0,len(lines)):
15          lines[i] = lines[i].replace("\n", "")
16          if len(lines[i])>200:
17              csvwriter.writerow([incremental, lines[i]])
18              incremental += 1
19      inputFile.close()
20  
21  out_csv.close()
```

Figure 8.1: Script used to clean the text in the NLP (Natural Language Processing) dataset

The script 8.2 (part 1) and 8.3 (part 2) was used to generate the bibliographies based on the articles present in the JSON file and put them inside, in this way the reference id in the bibliography is always coherent with the data inside the collection.
The same script was used also to save inside every author (in the authors JSON file) the preview of the articles he wrote as sub-documents. The goal is about speeding up the process of obtaining the list of the articles written by an author.

```
import json
import random
#from pprint import pprint
#import sys

main_json = open('main.json', encoding="UTF-8")
main_data = json.load(main_json)
#pprint (main_data)
main_json.close()

authors_pub = {}
doc_preview = {}

def sostisuisci_tags(articolo):
    lista_tags = [t['tag'] for t in articolo['tags']]
    res = [*set(lista_tags)]
    articolo['tags'] = res


#find all publications for each author
for doc in main_data:
    for author in doc["authors"]:
        name = author["name"]

        if name not in authors_pub: #insert the author in the dictionary if not present
            authors_pub[name]=[]

        temp ={"title" : doc["title"], "date" : doc["date"], "doc_id" : doc["doc_id"]}
        authors_pub[name].append(temp) #insert the document preview in the dictionary


#create a preview for all the documents
for doc in main_data:
    id = doc["doc_id"]
    doc_preview[id] = {"ref_id" : id, "title" : doc["title"], "date" : doc["date"]}


#write the article data in the final author JSON
author_json = open('author.json', encoding="UTF-8")
author_data = json.load(author_json)
```

Figure 8.2: Script used to create bibliography and article preview (Part 1)

```
41  author_json.close()
42
43  for author in author_data:
44      name = author["name"]
45
46      if name in authors_pub:
47          author["documents"] = authors_pub[name]
48
49  aut_output = json.dumps(author_data, indent=1, ensure_ascii=False)
50
51  with open("final_aut.json","w", encoding="UTF-8") as aut_file: #write the output file
52      aut_file.write(aut_output)
53
54
55  #generate bibliograpy references and write the final JSON for articles
56  for article in main_data:
57      num_citation = random.randrange(0,8)
58
59      bib_list = []
60      while len(bib_list) < num_citation:
61          id = random.randrange(1, len(main_data))
62          if id != article["doc_id"]:
63              bib_list.append(doc_preview[id])
64
65      if num_citation != 0:
66          article["bibliography"] = bib_list
67
68      sostisuisci_tags(article)
69
70
71  main_output = json.dumps(main_data, indent=1, ensure_ascii=False)
72
73
74  with open("final_main.json","w", encoding="UTF-8") as main_file: #write the output file
75      main_file.write(main_output)
```

Figure 8.3: Script used to create bibliography and article preview (Part 2)

8.2. Upload Process

At the end of the process there were created two collections in MongoDB: articles and authors in which have been imported the two JSON files generated in the previous steps using the apposite command button in the database graphic interface.
A cloud database is used to work on it together and also some test collections were created to write the commands without affecting the original data.

9 | Queries and Commands

9.1. Command 1

All elements with #cloudengineerjobs or #cloudengineering tag inside or with the "IWBS Report" journal are updated by adding the more general #cloud tag in the tags array and an attribute named "cloud" with the value of TRUE.

```
db.articles.updateMany(
    {$or : [{"tags": "#cloudengineerjobs"},
        {"tags" : "#cloudengineering"},
        {"journal": "IWBS Report"}]},
        {$push: {"tags": "#cloud"} ,
        $set: {"cloud" : true}})
```

9.2. Command 2

Insert a new article with a recent date, not in cloud and with some references to other articles in the DB and to the authors contained in the authors collection.

```
db.articles.insertOne(
 {
    doc_id:501,
    title: "John's Tech",
    abstract: "I prefer English than Latin",
    tags: ["#techman", "#notcloud"],
    url:" https://John.it/home",
    authors: [
    { "id": 368, "name": "Douglas Walton"},
    { "id": 234, "name": "Michael Schmitt 0002" }],
    journal: "Research Report / RJ / IBM / San Jose, California",
    volume: "1",
```

```
date: ISODate('2022-11-14T15:10:00.000Z'),
text_data: {
"title": "sapien a libero nam dui proin leo odio porttitor id consequat
in consequat ut nulla sed accumsan felis ut",
"image": {
  "url": "http://dummyimage.com/540x1109.png/5fa2dd/ffffff",
  "caption": "vel enim sit amet nunc viverra dapibus nulla suscipit
  ligula in lacus curabitur at ipsum ac tellus semper interdum mauris"
},
"section_array": [
 {
    "title": "primis in faucibus orci luctus et ultrices posuere
    cubilia curae mauris viverra diam vitae quam suspendisse potenti
    nullam porttitor lacus",
    "text": "They complied, and the three went on to the house where
    he lodged.  Whenthey turned into a narrow street, Stephen glanced
    at his window with adread that always haunted his desolate home;
    but it was open, as he hadleft it, and no one was there.  The evil
    spirit of his life had flittedaway again, months ago, and he had
    heard no more of her since.  The onlyevidence of her last return now,
    were the scantier moveables in his room,and the grayer hair upon
    his head.",
    "image": {
      "url": "http://dummyimage.com/1432x2591.png/cc0000/ffffff",
      "caption": "in magna bibendum imperdiet nullam orci pede venenatis
      non sodales sed tincidunt eu"
    },
    "sections": [
      {
        "title": "gravida sem praesent id massa id nisl venenatis lacinia
        aenean sit amet justo morbi ut odio cras",
        "text": "‘‘My dear, it’s the worst meal I’ve sat down to for years,’’
        said herhostess; ‘‘that last dish tasted principally of red pepper
        and wet toast.I’m awfully sorry.  Is anything the matter in the
        kitchen, Pellin?’’ sheasked of the attendant maid.",
        "image": {
          "url": "http://dummyimage.com/1196x1384.png/cc0000/ffffff",
```

```
      "caption": "a odio in hac habitasse platea dictumst maecenas ut
      massa quis augue luctus tincidunt nulla mollis molestie lorem
      quisque ut"
    }
  },
  {
    "title": "eros elementum pellentesque quisque porta volutpat
    erat quisque erat eros viverra eget congue eget semper rutrum",
    "text": "‘‘Well, old Bertram Kneyght is over in England just now
    from theArgentine.  He’s a kind of distant cousin of my mother’s,
    and soenormously rich that we’ve never let the relationship drop
    out of sight.Even if we don’t see him or hear from him for years
    he is always CousinBertram when he does turn up.  I can’t say
    he’s ever been of much soliduse to us, but yesterday the subject
    of my birthday cropped up, and heasked me to let him know what
    I wanted for a present.’’",
    "image": {
      "url": "http://dummyimage.com/1636x653.png/cc0000/ffffff",
      "caption": "vulputate nonummy maecenas tincidunt lacus at
      velit vivamus vel nulla eget eros elementum pellentesque
      quisque porta volutpat erat quisque erat"
    } },
  {
    "title": "vivamus metus arcu adipiscing molestie hendrerit
    at vulputate vitae nisl aenean lectus pellentesque eget",
    "text": "‘‘She was so good,’’ continued the bachelor, ‘‘that
    she won several medalsfor goodness, which she always wore,
    pinned on to her dress.  There was amedal for obedience,
    another medal for punctuality, and a third for goodbehaviour.
    They were large metal medals and they clicked against oneanother
    as she walked.  No other child in the town where she lived
    had asmany as three medals, so everybody knew that she must
    be an extra goodchild.’’",
    "image": {
      "url": "http://dummyimage.com/1723x1947.png/ff4444/ffffff",
      "caption": "elementum ligula vehicula consequat morbi a ipsum
      integer a nibh in quis justo maecenas rhoncus"
```

```
      } },
    {
      "title": "tristique fusce congue diam id ornare imperdiet sapien
      urna pretium nisl ut volutpat sapien arcu sed augue",
      "text": "He pushed the door open as he called to her, but did
      not return into theroom, or wait to be lighted down the narrow
      stairs. He was at the bottomwhen she began to descend, and was
      in the street before she could takehis arm.",
      "image": {
        "url": "http://dummyimage.com/1164x2963.png/dddddd/000000",
        "caption": "purus eu magna vulputate luctus cum sociis natoque
        penatibus et magnis dis parturient montes nascetur ridiculus"
      } } ] },
{
  "title": "vel augue vestibulum rutrum rutrum neque aenean auctor
  gravida sem praesent id massa",
  "text": "‘I have not so much as the slightest predilection left.
  I assure you Iattach not the least importance to any opinions.
  The result of thevarieties of boredom I have undergone, is a
  conviction (unless convictionis too industrious a word for the
  lazy sentiment I entertain on thesubject), that any set of ideas
  will do just as much good as any otherset, and just as much harm
  as any other set.  There’s an English familywith a charming Italian
  motto.  What will be, will be.  It’s the onlytruth going!’",
  "image": {
    "url": "http://dummyimage.com/990x1482.png/cc0000/ffffff",
    "caption": "rhoncus aliquet pulvinar sed nisl nunc rhoncus dui vel
    sem sed sagittis nam congue risus semper porta volutpat quam pede"
  },
  "sections": [
    {
      "title": "libero nullam sit amet turpis elementum ligula vehicula
      consequat morbi a ipsum integer a nibh in",
      "text": "He could do no less than ask Tom up; and Tom could do
      no less than go up.What with a cooling drink adapted to the
      weather, but not so weak ascool; and what with a rarer tobacco
      than was to be bought in those parts;Tom was soon in a highly
```

```
          free and easy state at his end of the sofa, andmore than ever
          disposed to admire his new friend at the other end.",
          "image": {
            "url": "http://dummyimage.com/1497x1089.png/ff4444/ffffff",
            "caption": "mi nulla ac enim in tempor turpis nec euismod
            scelerisque quam turpis adipiscing lorem vitae mattis nibh"
          } },
        {
          "title": "non lectus aliquam sit amet diam in magna bibendum
          imperdiet nullam orci pede venenatis non sodales sed
          tincidunt eu",
          "text": "‘Oh, my friends and fellow-men!’ said Slackbridge
          then, shaking his headwith violent scorn, ‘I do not wonder
          that you, the prostrate sons oflabour, are incredulous of
          the existence of such a man.  But he who soldhis birthright
          for a mess of pottage existed, and Judas Iscariot existed,
          and Castlereagh existed, and this man exists!’",
          "image": {
            "url": "http://dummyimage.com/751x2458.png/5fa2dd/ffffff",
            "caption": "nam ultrices libero non mattis pulvinar nulla
            pede ullamcorper augue a suscipit nulla elit ac nulla sed
            vel enim"
          } },
        {
          "title": "quis libero nullam sit amet turpis elementum ligula
          vehicula consequat morbi a ipsum integer a nibh",
          "text": "‘Thou changest me from bad to good.  Thou mak’st me
          humbly wishfo’ to bemore like thee, and fearfo’ to lose thee
          when this life is ower, and a’the muddle cleared awa’. Thou’rt
          an Angel; it may be, thou hast saved mysoul alive!’",
          "image": {
            "url": "http://dummyimage.com/1954x1097.png/dddddd/000000",
            "caption": "tempor turpis nec euismod scelerisque quam turpis
            adipiscing lorem vitae mattis nibh ligula nec"
          } } ] } ] },
  bibliography: [
    {
```

```
      "ref_id": 396,
      "title": "Generalized Coloring for Tree-like Graphs",
      "date": {
        "$date": {
          "$numberLong": "1405973512000"
        } } },
      {
      "ref_id": 236,
      "title": "SQL/Data System, General Information Manual.",
      "date": {
        "$date": {
          "$numberLong": "1582437539000"
        } } } ],
  cloud : false
})
```

9.3. Command 3

This command updates the articles that have the tag "#information" and adds the article with id 250 in bibliography if the articles we are updating already don't contain doc_id and ref_id equal to 250 which is the document we are trying to insert. In this way we are avoiding adding the article in its own bibliography or to have the same articles twice if it already exists in bibliography. In this scenario, we are assuming that the articles we are adding in the bibliography are already contained in the db.

```
db.articles.updateMany(
    {
        "doc_id":{$ne:250},
        "bibliography.ref_id":{$ne:250},
        "tags":"#information"},
        {‘‘$push’’: {bibliograpy: { ref_id: 250, ‘‘title’’:
            " Die Sloppy/Referenzielle Ambiguität: Bildungssyntax oder
            Diskursrepräsentationstheorie?",
            ‘‘Date": new ISODate("1998-08-16") } } } )
```

9.4. Command 4

This command deletes all the authors that have not any written article in the db.

```
db.authors.deleteMany({
    "documents":{"$exists":false}
})
```

9.5. Command 5

This query delete all documents that have the tag #business or #management published before 1980.

```
db.articles.deleteMany({
    $and: [
        {tags: {$in: [ '#business',  '#management']}},
        {date: { $lte: ISODate('1980-01-01T00:00:00.000Z')}}]
})
```

9.6. Command 6

This query remove an article with a certain title ('Towards a Transaction Management System for DOM.') from every journal.

```
db.articles.updateOne(
    {title: 'Towards a Transaction Management System for DOM.'},
    {$unset : {journal:""}}
)
```

9.7. Command 7

This command deletes an article between the ones that are too small(considering the number of pages) or too old(considering the publication date).
In the DB the smallest is with 61 pages and the oldest was published in 1994.

```
db.articles.deleteOne({
    $or : [{pages:{$lte : 65}},
    {date:{$lte: ISODate('1995-01-01T00:00:00.000Z')}}]
})
```

9.8. Query 1

The query returns the titles of the top 5 articles ordered by date, from the newest to the oldest. Each article needs to have:

- 2 or more authors that wrote at least 3 articles each
- An image on the cover
- A date after 01/01/2000
- At least 100 pages
- 3 paragraphs, at least one image hyperlinked in one of the paragraphs

Complexity: Multiple filtering conditions (also on internal subdocs), aggregation, unwind, ref.

```
db.articles.aggregate([
  { $match: {
      date: { $gte: ISODate('2000-01-01T00:00:00.000Z') },
      'text_data.image': { $exists: true },
      pages: { $gte: 100 },
      'text_data.section_array': { $size: 3 } } },
  { $lookup: {
      from: 'authors',
      localField: 'authors.id',
      foreignField: 'author_id',
      as: 'authorsDetailed' } },
  { $unwind: { path: '$authorsDetailed' } },
  { $project: { doc_id: 1, authors: 1, title: 1,
        numOfDocsXAuthor: {
            $size: '$authorsDetailed.documents'},
        text_data: 1,
        date: 1 } },
  { $match: { numOfDocsXAuthor: { $gte: 3 } } },
  { $group: { _id: '$doc_id', title: {
        $first: '$title' }, date: { $first: '$date' },
        text_data: { $first: '$text_data' },
        authorFilteredPerDocument: { $sum: 1 } } },
```

```
  { $match: { authorFilteredPerDocument: { $gte: 2 } } },
  { $unwind: { path: '$text_data.section_array' } },
  { $match: { 'text_data.section_array.image.url': {
        $exists: true } } },
  { $group: { _id: '$_id', doc_id: { $first: '$_id' },
        title: { $first: '$title' },
        date: { $first: '$date' },
        numImagesWithHyperlink: { $sum: 1 } } },
  { $match: { numImagesWithHyperlink: { $gte: 1 } } },
  { $sort: { date: -1 } },
  { $project: { doc_id: 1, title: 1,
        date: { $dateToString: { date: '$date' } } } },
  { $limit: 5}
])
```

Results:


```
[{
  "_id": 251,
  "doc_id": 251,
  "title": "Standards für das Programmieren in Prolog für LILOG, Version 3.3",
  "date": "2021-06-19T05:39:43.000Z"
},{
  "_id": 21,
  "doc_id": 21,
  "title": "Die Repräsentation räumlichen Wissens und die Behandlung von
  Einbettungsproblemen mit Quadtreedepiktionen",
  "date": "2020-01-16T03:01:44.000Z"
},{
  "_id": 254,
  "doc_id": 254,
  "title": "Using a Structured English Query Language as a Data Definition
  Facility.",
  "date": "2019-08-27T21:23:26.000Z"
},{
  "_id": 489,
  "doc_id": 489,
  "title": "On the Expressiveness of Tree Set Operators",
```

```
  "date": "2019-06-08T00:52:34.000Z"
},{
  "_id": 179,
  "doc_id": 179,
  "title": "EFTA Query Processing in LILOG-DB",
  "date": "2019-05-22T17:21:23.000Z"
}]
```


9.9. Query 2

This query returns the 5 articles of the journal "LILOG-Report" written between 2000 and 2006 by the largest number of authors. The result is sorted by the number of authors and the publication date in descending order.
Complexity: Multiple filtering conditions, aggregation.

```
db.articles.aggregate([
    {"$match":{"journal":"LILOG-Report", "date": {$gte: ISODate('2000-01-01'),
        $lt: ISODate('2007-01-01')}}},
    {"$project":{"title":"$title", "number of authors": {"$size" : "$authors"},
        "date of publication" : "$date"}},
    {"$sort": { "number of authors" : -1,"date of publication":-1}},
    {"$limit":5}
])
```

Results:


```
[{ _id: ObjectId("637a5e477cc4275cf6f3e1a4"),
title: 'Datenbankunterstützung für wissensbasierte Systeme',
'number of authors': 5,
'date of publication': 2005-10-30T20:29:07.000Z },

{ _id: ObjectId("637a5e477cc4275cf6f3e1fb"),
title: 'A Conceptual Model for Time',
'number of authors': 5,
'date of publication': 2004-05-07T23:47:03.000Z },

{ _id: ObjectId("637a5e477cc4275cf6f3e1f2"),
title: 'Mentale Bilder und Wegbedeutungen',
```

```
'number of authors': 4,
'date of publication': 2001-09-01T10:59:15.000Z },

{ _id: ObjectId("637a5e477cc4275cf6f3e201"),
title: 'A Feature Logic with Subsorts',
'number of authors': 3,
'date of publication': 2005-07-01T11:09:37.000Z },

{ _id: ObjectId("637a5e477cc4275cf6f3e16c"),
title: 'Mengenorientierte Auswertung von Anfragen in der
Logikprogrammiersprache PROLOG',
'number of authors': 3,
'date of publication': 2002-03-18T17:26:24.000Z }]
```

9.10. Query 3

This query returns the articles which have the tag #tech or belong to the journal "GTE Laboratories Incorpored" and have at least in section_array one section with size 2 (so 2 subsections). The results that are returned are 4 and are sorted in alphabetical order. Complexity: Multiple filtering conditions (also on internal subdocs).

```
db.articles.find(
    {"$and":[
        {"$or":[
            {"journal":"GTE Laboratories Incorpored"},
            {"tags":"#tech"}]},
        {"text_data.section_array":
            {"$elemMatch":{"sections":{"$size":2}}}}]},
    {"title":1,"journal":1, "tags":1}
).sort({"title":1}).limit(4)
```

Results:

```
[{ _id: ObjectId("637a916884bb1e6096b606fa"),
  title: 'A Logical Operational Semantics of Full Prolog Part III: Built-in
  Predicates for Files, Terms, Arithmetic and Input-Output',
  tags: [ '#tech', '#inforubaya', '#science', '#datadciencecourse' ],
  journal: 'IWBS Report' },
```

```
{ _id: ObjectId("637a916884bb1e6096b6084f"),
  title: 'Computational Convexity',
  tags: [ '#bigdataanalytics', '#tech', '#bigdatacourses' ],
  journal: 'Universität Trier, Mathematik/Informatik, Forschungsbericht' },

{ _id: ObjectId("637a916884bb1e6096b60698"),
  title: 'Computational aspects of three-valued logic',
  tags:
   [ '#cloudenginetech',
     '#tech',
     '#productmanagementcourse',
     '#informationtechnology' ],
  journal: 'LILOG-Report' },

{ _id: ObjectId("637a916884bb1e6096b607ce"),
  title: 'Convergence of Generalized Supermartingales with Continous Parameter',
  tags: [ '#informatika', '#tech' ],
  journal: 'Universität Trier, Mathematik/Informatik, Forschungsbericht' }]
```

9.11. Query 4

This query returns the articles (with all the filtered fields) that have at least one document of their bibliography published after 2018 (included) and that one of the authors is affiliated to Universidade do Estado do Rio de Janeiro.
Complexity: Multiple filtering conditions through refs, aggregation, also on internal subdocs.

```
db.articles.aggregate([
    {"$match":{"bibliography": {"$exists" : true },
      "bibliography.date": {"$gte" : ISODate('2018-01-01') } } },
    {"$lookup":{
        from:"authors",
        localField:"authors.id",
        foreignField:"author_id",
        as:"authorsDetailed"}},
    {"$match":{
        "authorsDetailed.affiliation": "Universidade do Estado do Rio
```

```
        de Janeiro"}},
    {"$project":{ "title":1,"bibliography.date":1,
        "authorsDetailed.affiliation": 1}}
])
```

Results:

```
[{ _id: ObjectId("63727cd017e0d1dfd9c10ef6"),
title: 'Numerical Solution of a Semilinear Parabolic Control Problem',
bibliography:
[ { date: 2008-07-29T20:20:08.000Z },
{ date: 2021-02-10T23:37:46.000Z },
{ date: 2013-09-11T16:02:31.000Z } ],
authorsDetailed:
[ { affiliation: 'Faculté Polytechnique de Mons' },
{ affiliation: 'Universidade do Estado do Rio de Janeiro' },
{ affiliation: 'East-Siberian State University' },
{ affiliation: 'Southern Polytechnic State Univerisity' },
{ affiliation: 'International University of Kyrgyzstan' } ] }]
```

9.12. Query 5

This query finds the names and affiliations of the top 5 authors that wrote at least 5 articles. The results are shown ordered descending by longest biography.
Complexity: Multiple filtering conditions, aggregation.

```
db.authors.aggregate([
    { $match: { documents: { $exists: true } } },
    { $project: {
        name: 1,
        affiliation: 1,
        bio_lenght: { $strLenCP: '$biography' },
        num_articles: { $size: '$documents' } } },
    { $match: { num_articles: { $gte: 5 } } },
    { $sort: { bio_lenght: -1 } },
    { $limit: 5 }
])
```

Results:

```
[{ _id: ObjectId("637a5e5a7cc4275cf6f3e391"),
name: 'Holger M. Wittmann',
affiliation: 'Universität Koblenz-Landau',
bio_lenght: 970,
num_articles: 7 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e49b"),
name: 'Wessel Reijers',
affiliation: 'Ohio State University',
bio_lenght: 819,
num_articles: 5 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e3b2"),
name: 'Bernhard Nebel',
affiliation: 'Higher School of University and Advanced Studies Pisa',
bio_lenght: 812,
num_articles: 5 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e375"),
name: 'Claudia Mrotzek',
affiliation: 'Arab Open University',
bio_lenght: 695,
num_articles: 5 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e70e"),
name: 'Maryam Ronagh',
affiliation: 'Vilnius University',
bio_lenght: 651,
num_articles: 5 }]
```

9.13. Query 6

This query returns 5 articles that were written after 2021 (included) or have less than 50 pages(included) or having the tag #software and at least one element of the bibliography

published before 1995.
Complexity: Multiple filtering conditions, also on internal subdocs.

```
db.articles.find(
    {"$and":[
        {"$or":[ {"date":{"$gte":ISODate('2021-01-01')}},
            {"pages":{"$lte":50}}, {"tags":"#software"}]},
        {"bibliography.date": {"$lt":new ISODate('1995-01-01')}}]},
    {"title":1, "date":1, "pages":1, "tags":1, "bibliography.date":1}
).sort({"title":1}).limit(5)
```

Results:

```
[{ _id: ObjectId("63727cd017e0d1dfd9c10f02"),
title: 'On the Queue-Length-Distributions of the Ek/MX/1 Queue and of the
Ek/GE/1 Queue under Repetitive-Service Blocking',
tags:
[ '#phythoncode',
'#tech',
'#software',
'#healthinformaticsprofessional' ],
date: 1998-11-09T15:56:51.000Z,
pages: 255,
bibliography:
[ { date: 1994-04-26T06:39:31.000Z },
{ date: 2016-08-29T16:10:55.000Z } ] },

{ _id: ObjectId("63727cd017e0d1dfd9c10f53"),
title: 'On the Rareness of Disturbed Martingales in the Class of all Uniformly
L1-bounded Stochastic Processes',
tags: [ '#datascience' ],
date: 2021-06-16T10:15:52.000Z,
pages: 956,
bibliography:
[ { date: 2016-08-16T13:05:53.000Z },
{ date: 2002-02-10T12:35:52.000Z },
{ date: 1994-02-23T22:23:53.000Z },
{ date: 2004-03-02T23:54:29.000Z },
```

```
{ date: 2018-09-27T12:23:52.000Z },
{ date: 1994-02-23T22:23:53.000Z },
{ date: 1996-09-13T15:35:56.000Z } ] },

{ _id: ObjectId("63727cd017e0d1dfd9c10f3d"),
title: 'On the Solution of a Two Ball Trust Region Subproblem',
tags: [ '#software', '#computerscience', '#healthcare', '#information' ],
date: 2014-09-30T01:52:13.000Z,
pages: 302,
bibliography:
[ { date: 1999-09-10T08:55:09.000Z },
{ date: 2000-11-14T13:46:50.000Z },
{ date: 1994-07-10T02:59:15.000Z } ] },

{ _id: ObjectId("63727cd017e0d1dfd9c10f84"),
title: 'Separating Counting Communication Complexity Classes',
tags: [ '#coding' ],
date: 2022-04-01T06:30:22.000Z,
pages: 802,
bibliography:
[ { date: 2016-06-22T18:27:56.000Z },
{ date: 1994-08-30T02:07:39.000Z },
{ date: 1997-10-22T19:24:38.000Z },
{ date: 2003-04-18T22:36:33.000Z },
{ date: 2018-09-28T13:18:43.000Z },
{ date: 2003-01-12T12:25:14.000Z } ] },

{ _id: ObjectId("63727cd017e0d1dfd9c10f25"),
title: 'Some Heuristics for Generating Tree-like FBDD Types',
tags: [ '#pharmd', '#healthinformaticsprofessional' ],
date: 2022-09-26T06:54:16.000Z,
pages: 845,
bibliography: [ { date: 1994-06-10T21:05:47.000Z } ] } ]
```

9.14. Query 7

This query finds the title and publication date of all the articles published between 01/01/2000 and 01/01/2007 which cite the article with doc_id: 304. The results are ordered by descending date.
Complexity: Multiple filtering conditions, aggregation (also on internal subdocs).

```
db.articles.aggregate([{
    $match: {
        date: {
        $gte: ISODate('2000-01-01T00:00:00.000Z'),
        $lt: ISODate('2007-01-01T00:00:00.000Z') },
        'bibliography.ref_id': 304 } },
    { $project: { title: 1, date: 1 } },
    { $sort: { date: -1 } }
])
```

Results:

```
{ _id: ObjectId("637a916884bb1e6096b606f1"),
  title: 'Stepwise Software Development with Algebraic Specifications',
  date: 2005-03-29T09:19:28.000Z }

{ _id: ObjectId("637a916884bb1e6096b60675"),
  title: 'An Algebraic Characterization of STUF',
  date: 2003-02-27T00:22:08.000Z }

{ _id: ObjectId("637a916884bb1e6096b606c0"),
  title: 'A Model Elimination Calculus for Generalized Clauses',
  date: 2002-03-28T04:03:16.000Z }
```

9.15. Query 8

This query finds the names and affiliations of the top 5 authors that wrote at least 5 articles. The results are shown ordered descending by longest biography.
Complexity: Multiple filtering conditions, aggregation.

```
db.authors.aggregate([
```

```
    { $match: { "documents.4": { $exists: true } }},
    { $project: { name: 1, affiliation: 1,
        bio_lenght: { $strLenCP: '$biography' } } },
    { $sort: { bio_lenght: -1 } },
    { $limit: 5 }
])
```

Results:

```
[{ _id: ObjectId("637a5e5a7cc4275cf6f3e391"),
name: 'Holger M. Wittmann',
affiliation: 'Universität Koblenz-Landau',
bio_lenght: 970,
num_articles: 7 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e49b"),
name: 'Wessel Reijers',
affiliation: 'Ohio State University',
bio_lenght: 819,
num_articles: 5 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e3b2"),
name: 'Bernhard Nebel',
affiliation: 'Higher School of University and Advanced Studies Pisa',
bio_lenght: 812,
num_articles: 5 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e375"),
name: 'Claudia Mrotzek',
affiliation: 'Arab Open University',
bio_lenght: 695,
num_articles: 5 },

{ _id: ObjectId("637a5e5a7cc4275cf6f3e70e"),
name: 'Maryam Ronagh',
affiliation: 'Vilnius University',
bio_lenght: 651,
```

```
num_articles: 5 }]
```

9.16. Query 9

This query takes all documents with more than 500 pages and date less than 2013-04-11, there is a sorting by the pages in ascending order, an unwind by referenced articles (bibliography) and takes all those with date greater than 2013-04-11 returning only the title of the ref and the doc_id of the source document. It shows only the first 10 results. Complexity: Filtering (also con subdocs), aggregation, unwind, refs.

```
db.articles.aggregate([
    { $match: { $and: [
        { pages: { $gte: 500 } },
    { date: { $lt: ISODate('2013-04-11T00:00:00.000Z') } } ] } },
    { $sort: { pages: 1 } },
    { $unwind: { path: '$bibliography' } },
    { $match: { 'bibliography.date': {
        $gte: ISODate('2013-04-11T00:00:00.000Z') } } },
    { $project: { 'bibliography.title': 1, doc_id: 1,
        _id: 0 } },
    { $limit: 10 }
])
```

Results:

```
[{ doc_id: 29,
    bibliography: { title:'Datenbankkonzepte für wissensbasierte
    Systeme'}},

{ doc_id: 280,
    bibliography: { title: 'Temporales Schließen in einem
    natürlichsprachlichen System' } },

{ doc_id: 383,
    bibliography: { title: 'Chart-Parsing of Unification-Based
    Grammars with ID/LP-Rules' } },

{ doc_id: 383,
```

```
    bibliography: { title: 'Locating Matches of Tree Patterns
    in Forests' } },

{ doc_id: 383,
    bibliography: { title: 'Object Oriented DBMS as a Generalization
    of Relational DBMS' } },

{ doc_id: 383,
    bibliography: { title: 'Modals and the Scope of Negation, On the
    Treatment of Optionality in HPSG, Some Properties of Negation in
    German.' } },

{ doc_id: 383,
    bibliography: { title: 'Attributive Concept Descriptions with Unions
    and Complements' } },

{ doc_id: 99,
    bibliography: { title: 'Functional Quantization and Small Ball
    Probabilities for Gaussian Processes' } },

{ doc_id: 440,
    bibliography: { title: 'Object Data Language Facilities for
    Multimedia Data Types.' } },

{ doc_id: 440,
    bibliography: { title: 'Scheduling and Related Graph Theoretical
    Problems' } }]
```

9.17. Query 10

This query retrieve 6 documents with more authors among the ones that were published after 2005 with the tag "#datascience" or "#programming" or publicated before 2005 but having the tag "#computer".
Complexity: Multiple filtering conditions, aggregation.

```
db.articles.aggregate([
    { $match:{ $or:
```

```
                [
                    {$and:[{ tags:{$in:["#datascience","#programming"]}},
                    {date:{"$gte":new Date("2005-01-01")}}]},
                    {$and:[{tags:"#computer"},
                         { date:{"$lt":new Date("2005-01-01")} }]}
                ] } },
   { $project:{ title:1, tags:1, date:1,
        numAuthors: { $cond: { if: { $isArray: "$authors" }, then:
            { $size: "$authors" }, else: "NA"} } } },
   { $sort : { numAuthors : -1 } },
   { $limit:6 }
])
```

Results:

```
[{ _id: ObjectId("637a916884bb1e6096b60683"),
    title: 'Attribute Inheritance Implemented on Top of a Relational
    Database System', tags: [ '#phythonprogramming', '#programming' ],
    date: 2021-07-17T13:36:53.000Z,
    numAuthors: 5 },

{ _id: ObjectId("637a916884bb1e6096b60756"),
title: 'On Structuring Domain-Specific Knowledge',
tags: [ '#teknikinformatika', '#programming' ],
date: 2010-01-26T03:49:58.000Z,
numAuthors: 5 },

{ _id: ObjectId("637a916884bb1e6096b607d9"),
title: 'On the Rareness of Disturbed Martingales in the Class of all Uniformly
L1-bounded Stochastic Processes',
tags: [ '#datascience' ],
date: 2021-06-16T10:15:52.000Z,
numAuthors: 5 },

{ _id: ObjectId("637a916884bb1e6096b60809"),
title: 'The complexity of optimal small policies',
tags: [ '#programming' ],
```

```
date: 2007-12-10T09:13:42.000Z,
numAuthors: 5 },

{ _id: ObjectId("637a916884bb1e6096b60706"),
title: 'The Design and Implementation of TLPROLOG (V. 2.0) - Intermediate Report',
tags: [ '#computer', '#informatik', '#cloudengineer', '#informatika' ],
date: 1998-04-06T09:08:09.000Z,
numAuthors: 5 },

{ _id: ObjectId("637a916884bb1e6096b60672"),
title: 'Die Repräsentation räumlichen Wissens und die Behandlung von
Einbettungsproblemen mit Quadtreedepiktionen',
tags:
[ '#datascience',
'#webdeveloper',
'#coding',
'#nurseinformatics' ],
date: 2020-01-16T03:01:44.000Z,
numAuthors: 4 }]
```

9.18. Query 11

This query retrieve a ranking of the years, from 2005, with more articles published having among their tags one of the following tags: "#datascience", "#programming", "#programmer", "#phythonprogramming", "#aws", "#cloudengineercourses"; sometimes the results are in a different order because of the similar values in the sorting.
Complexity: Multiple filtering conditions, aggregation.

```
db.articles.aggregate([
   {$match:{
          $and:[{ tags:{$in:["#datascience", "#programming", "#programmer",
          "#phythonprogramming", "#aws", "#cloudengineercourses"]}},
          {date:{"$gte":new Date("2005-01-01")}}]}},
   { $group:{ _id:{$year:"$date" }, count:{$sum:1}}},
   { $project: {
        count:1,
        _id:1}},
```

```
{ $sort : { count : -1 }},
{ $limit : 15 }] )
```

Results (the group _id is the year):

```
[{ _id: 2007, count: 8 },
{ _id: 2011, count: 7 },
{ _id: 2013, count: 7 },
{ _id: 2019, count: 7 },
{ _id: 2005, count: 6 },
{ _id: 2018, count: 6 },
{ _id: 2021, count: 6 },
{ _id: 2014, count: 5 },
{ _id: 2022, count: 5 },
{ _id: 2020, count: 4 },
{ _id: 2009, count: 4 },
{ _id: 2016, count: 3 },
{ _id: 2010, count: 3 },
{ _id: 2015, count: 3 },
{ _id: 2006, count: 3 }]
```

9.19. Query 12

This query retrieve for every year between 2005 and 2021 the number of articles, if published, with the tag #cloudengineers or #datascience.
Complexity: Multiple filtering condition, aggregation, unwind, refs.

```
db.articles.aggregate([
    { $match: {
        $and:[
        {date: {
            "$gte":new Date("2005-01-01"),
            "$lt":new Date("2021-01-01") } },
        {tags:{$in:["#cloudengineers",
            "#datascience"] } } ] } },
    { $unwind:"$tags" },
    { $group:{  _id:{year:{$year:"$date"} , tag:"$tags"}, count:{$sum:1} } },
    { $project: { _id:0, "tag":"$_id.tag",  "year":"$_id.year",  count:1 } },
```

```
    { $match: {tag:{$in:["#cloudengineers", "#datascience"]}} },
    { $sort : { "year" : -1,"count": -1 } },
    { $limit :13 }
])
```

Results:

```
[{ count: 1, tag: '#datascience', year: 2020 },
{ count: 1, tag: '#datascience', year: 2019 },
{ count: 3, tag: '#cloudengineers', year: 2018 },
{ count: 1, tag: '#datascience', year: 2018 },
{ count: 1, tag: '#cloudengineers', year: 2015 },
{ count: 1, tag: '#cloudengineers', year: 2014 },
{ count: 1, tag: '#cloudengineers', year: 2013 },
{ count: 3, tag: '#datascience', year: 2011 },
{ count: 1, tag: '#cloudengineers', year: 2011 },
{ count: 1, tag: '#datascience', year: 2008 },
{ count: 1, tag: '#cloudengineers', year: 2008 },
{ count: 1, tag: '#datascience', year: 2007 },
{ count: 1, tag: '#cloudengineers', year: 2006 }]
```

9.20. Query 13

This query retrieve the title of the top 10 articles, published after 1990, for number of citations having more than 100 pages.
Complexity: Multiple filtering conditions, unwind, aggregation.

```
db.articles.aggregate ([
    { $match: { $and:[{pages:{$gte: 100} },
              {date:{"$gte":new Date("1990-01-01") } } ] } },
    { $unwind: {
          path: '$bibliography',
          preserveNullAndEmptyArrays: false } },
    { $group: {
           _id: {
           id: '$bibliography.ref_id',
           title: '$bibliography.title' },
          num_Citation: { $sum: 1 } } },
```

```
    { $project: {
          _id: 0,
          title: '$_id.title',
          reference_Id: '$_id.id',
          num_Citation: 1 } },
    { $sort: { num_Citation: -1 } },
    { $limit: 10 }
])
```

Results:

```
[{ num_Citation: 11,
title: 'The Algebra of Feature Graph Specifications',
reference_Id: 235 },

{ num_Citation: 10,
title: 'A Fast and Robust Algorithm for a Class of Generalized
Fractional Programs',
reference_Id: 289 },

{ num_Citation: 9,
title: 'Block-Restricted Reordering - Extended Experiments',
reference_Id: 321 },

{ num_Citation: 9,
title: 'Zum Begriff "Argumentstruktur" im Rahmen generativer
worbildungstheoretischer Ansätze',
reference_Id: 113 },

{ num_Citation: 9,
title: 'Draft Report of the Logic Programming Language PROTOS-L',
reference_Id: 177 }

{ num_Citation: 8,
title: 'Inductive Counting below LOGSPACE',
reference_Id: 462 },
```

```
{ num_Citation: 8,
title: 'Ein Beispiel zur wissensbasierten Produktionsplanung mit
PROTOS-L.',
reference_Id: 112 },

{ num_Citation: 8,
title: 'Increasing Efficiency of Symbolic Model Checking by
Accelerating Dynamic Variable Reordering',
reference_Id: 318 },

{ num_Citation: 8,
title: 'Zur Verarbeitung von Lokalisierungsausdrücken in
einem hybriden System',
reference_Id: 159 },

{ num_Citation: 8,
title: 'EFTA: A Database Retrieval Algebra for Feature Terms',
reference_Id: 42 }]
```

9.21. Query 14

This query retrieve the top 5 authors with the major number of citations related to articles published after 1990 and with more than 100 pages.
Complexity: Multiple Filtering condition, aggregation (also on subdocs), through refs.

```
db.articles.aggregate ([
    { $match: {
        $and:[{pages:{$gte: 100} },
            {date:{"$gte":new Date("1990-01-01") } } ] } },
    { $unwind: {
          path: '$bibliography',
          preserveNullAndEmptyArrays: false } },
    { $group: {
           _id: {
           id: '$bibliography.ref_id',
           title: '$bibliography.title' },
           num_Citation: { $sum: 1 } } },
```

```
    { $lookup: {
          from: 'articles',
          localField: '_id.id',
          foreignField: 'doc_id',
          as: 'ArticleDetail' } },
    { $unwind: { path: '$ArticleDetail' } },
    { $unwind: { path: '$ArticleDetail.authors' } },
    { $group: {
           _id: {
           Authorid: '$ArticleDetail.authors.id',
           AuthorName: '$ArticleDetail.authors.name' },
          TotalCitation: { $sum: '$num_Citation' } } },
    { $project: {
          author: '$_id.AuthorName',
          id: '$_id.Authorid',
          Citations: '$TotalCitation',
          _id: 0 } },
    { $sort: { Citations: -1 } },
    { $limit: 5 }
])
```

Results:

```
[{ author: 'Rebecca Davnall', id: 672, Citations: 25 },
{ author: 'Erich Gehlen', id: 19, Citations: 22 },
{ author: 'Quinn Grundy', id: 936, Citations: 21 },
{ author: 'Holger M. Wittmann', id: 75, Citations: 20 },
{ author: 'Edoardo Datteri', id: 894, Citations: 20 }]
```

9.22. Query 15

This query retrieves the top 5 articles that have the greatest number of sub-section with an image, considering all articles having among the tags “#datascience", "#programming", "#programmer", "#phythonprogramming", "#aws", "#cloudengineercourses” and published after 2000.
Complexity: Multiple filtering condition (also on subdocs), aggregation, unwind.

```
db.articles.aggregate ([
```

```
    { $match: { $and: [
        { tags: { $in: [ '#datascience', '#programming',
                '#programmer', '#phythonprogramming', '#aws',
                '#cloudengineercourses' ] } },
        { date: { $gte: ISODate('2000-01-01T00:00:00.000Z') } } ] } },
    { $unwind: { path: '$text_data.section_array' } },
    { $unwind: { path: '$text_data.section_array.sections' } },
    { $match: { 'text_data.section_array.sections.image': {"$exists" : true }} },
    { $group: {
           _id: {
           articleTitle: '$title',
           articleId: '$doc_id' },
           NumSubsectionWithImage: { $sum: 1 } } },
    { $project: {
           _id: 0,
           title: '$_id.articleTitle',
           id: '$_id.articleId',
           NumSubsectionWithImage: 1 } },
    { $sort: { NumSubsectionWithImage: -1 } },
    { $limit: 5 }
])
```

Results:

```
[{
  "NumSubsectionsWithImages": 25,
  "title": "Constraint Decomposition Algorithms in Global Optimization",
  "id": 453
}, {
  " NumSubsectionsWithImages ": 25,
  "title": "Attribute Inheritance Implemented on Top of a Relational Database
  System",
  "id": 38
}, {
  " NumSubsectionsWithImages: 24,
  "title": "An Evaluation of Object-Oriented DBMS Developments: 1994 Edition.",
  "id": 3
```

```
}, {
  " NumSubsectionsWithImages: 24,
  "title": "Complexity Classes with Finite Acceptance Types",
  "id": 327
}, {
  "NumSubsectionsWithImages": 22,
  "title": "18. Workshop über Komplexitätstheorie, effiziente Algorithmen und
  Datenstrukturen, Universität Trier, 20. Oktober 1992 (Abstracts)",
  "id": 273
}]
```

9.23. Query 16

This query retrieve the biographies of all the authors of the article "Syntaktische Informationseinheiten zu deutschen Adjektiven".
Complexity: Multiple filtering condition (also on subdocs), aggregation, unwind.

```
db.articles.aggregate([
    { $match: {
        title: 'Syntaktische Informationseinheiten zu deutschen
        Adjektiven'} },
    { $unwind: { path: '$authors' } },
    { $lookup: {
        from: 'authors',
        localField: 'authors.id',
        foreignField: 'author_id',
        as: 'authorDetail'} },
    { $unwind: { path: '$authorDetail' } },
    { $project: {
        _id: 0,
        AuthorName: '$authorDetail.name',
        AuthorBio: '$authorDetail.biography'} }
])
```

Results:

```
{ AuthorName: 'Horst Schmitt',
AuthorBio: 'Duis aliquam convallis nunc. Proin at turpis a pede
```

posuere nonummy. Integer non velit.Donec diam neque, vestibulum eget, vulputate ut, ultrices vel, augue. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Donec pharetra, magna vestibulum aliquet ultrices, erat tortor sollicitudin mi, sit amet lobortis sapien sapien non mi. Integer ac neque.Duis bibendum. Morbi non quam nec dui luctus rutrum. Nulla tellus.In sagittis dui vel nisl. Duis ac nibh. Fusce lacus purus, aliquet at, feugiat non, pretium quis, lectus. Suspendisse potenti. In eleifend quam a odio. In hac habitasse platea dictumst.Maecenas ut massa quis augue luctus tincidunt. Nulla mollis molestie lorem. Quisque ut erat.' }

{ AuthorName: 'Helmut Seidl',
AuthorBio: 'Quisque id justo sit amet sapien dignissim vestibulum. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nulla dapibus dolor vel est. Donec odio justo, sollicitudin ut, suscipit a, feugiat et, eros. Vestibulum ac est lacinia nisi venenatis tristique. Fusce congue, diam id ornare imperdiet, sapien urna pretium nisl, ut volutpat sapien arcu sed augue. Aliquam erat volutpat. In congue. Etiam justo. Etiam pretium iaculis justo. In hac habitasse platea dictumst. Etiam faucibus cursus urna. Ut tellus. Nulla ut erat id mauris vulputate elementum. Nullam varius. Nulla facilisi.' }

{ AuthorName: 'Yue Liang Zheng',
AuthorBio: 'Integer tincidunt ante vel ipsum. Praesent blandit lacinia erat. Vestibulum sed magna at nunc commodo placerat. Praesent blandit. Nam nulla. Integer pede justo, lacinia eget, tincidunt eget, tempus vel, pede. Morbi porttitor lorem id ligula. Suspendisse ornare consequat lectus. In est risus, auctor sed, tristique in, tempus sit amet, sem. Fusce consequat. Nulla nisl. Nunc nisl. Duis bibendum, felis sed interdum venenatis, turpis enim blandit mi, in porttitor pede justo eu massa. Donec dapibus. Duis at velit eu est congue elementum. In hac habitasse platea dictumst. Morbi vestibulum, velit id pretium iaculis, diam erat fermentum justo, nec condimentum neque sapien placerat ante. Nulla justo. Aliquam quis turpis eget elit sodales scelerisque. Mauris sit amet eros. Suspendisse accumsan tortor quis turpis.' }

Part III

Third Delivery: Spark

10 | Introduction

10.1. Specification

The aim of the third delivery is to load the data on the Apache Spark distributed framework to perform analysis on them (using the Python pyspark library, but without using SQL language).

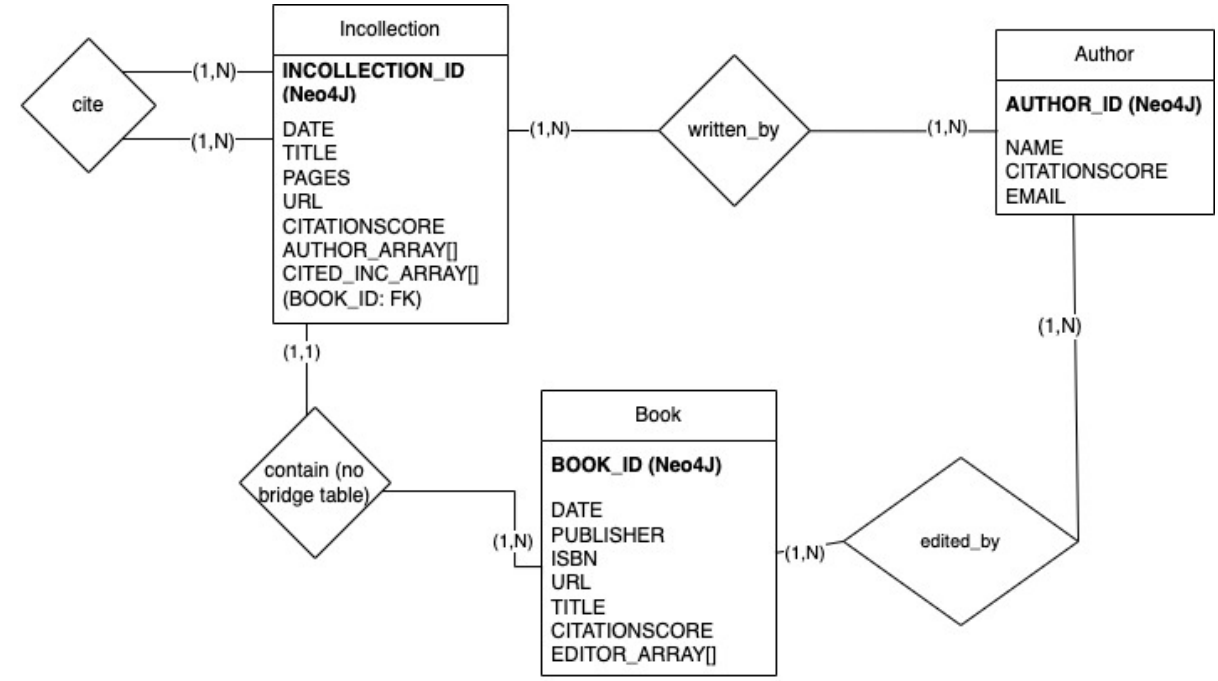
10.2. Hypothesis and assumptions

This part deploys Neo4J data, because there were already relationships and all the information needed. The data were pulled out of Neo4j by making the queries needed exporting the outputs in json (by directly exporting the csv the data was not complete), then the json files were converted into csv using an online tool and the csv were modified using Excel formatting the columns and removing/adding attributes (for example the email attribute was added to the authors by generating it coherently with their name, or some attributes were eliminated for simplicity).

11 | Document Description

11.1. Document schema

First, there was a reasoning on an ER diagram, then some changes was made to the schema to adapt it to the Spark database (for example using array of foreign keys instead of bridge tables for N to N relationships).



NB: TO AVOID REDUNDANCY (ERRATED IN THE RELATIONAL DBS) AND TO AVOID TOO MUCH COMPLEXITY, ONLY THE CITE OF THE INCOLLECTIONS ARE CONSIDERED AND THE BOOKS HAVE EDITORS (WHICH ARE IN REALITY THE SAME ENTITY AS THE AUTHORS SO AS TO HAVE CONSISTENCY WITH NEO4J AND ER)
N:N RELATIONSHIPS ARE NOT TREATED WITH BRIDGE TABLE AS IN A NORMAL RELATIONAL DATABASE BUT THEY ARE ADAPTED WITH FOREIGN KEYS ARRAYS (WHICH MAY ALSO BE EMPTY)

12 | Data Creation and Transformation

12.1. Pre-processing

A Python script has been used to convert the bridge tables in an array of keys and considered as attribute in the the more important between the two entities of the relation (to save space). There was a final format on the csv.txt files to have the files ready to be imported and to be configured on spark.

12.2. Upload Process

For convenience, since the csv file already got the headers, they have been directly imported into dataframes (and not into rdd to modify and convert them); later they have been managed to make each attribute of the right type (such as dates in type Date), everything has been tested by printing schema and some examples of tables.

Init part of importing

Two parts:

- **Connection to spark:** The first piece of code.
- **Importing and parsing of dataframes** The second piece of code.

[1]:
```
# Import the basic spark library
from pyspark.sql import SparkSession

# Create an entry point to the PySpark Application
spark = SparkSession.builder \
      .master("local") \
      .appName("ProjSMBUD") \
      .getOrCreate()
# master contains the URL of your remote spark instance or 'local'
```

•••

•[2]:
```
# Load DataFrames (we prefer directly them to the rdd, because we already had csv with headers)
#Configuring them with some examples to check the correctness (now commented before and after the casting)
df_incollection = spark.read.option("header", True).option("encoding", "ISO-8859-1").option("delimiter", ";")\
                  .csv("incollectionFormatted.csv.txt")
df_book = spark.read.option("header", True).option("encoding", "ISO-8859-1").option("delimiter", ";")\
                  .csv("bookFormatted.csv.txt")
df_author = spark.read.option("header", True).option("encoding", "ISO-8859-1").option("delimiter", ",")\
                  .csv("authorFormatted.csv.txt")

#Load bridge tables? (No, now they are in the entities), #they had already id string types, but now we don't have them
```

```
# We use datatypes from pyspark.sql.types
from pyspark.sql.types import StringType, DateType, FloatType, StructType, StructField, ArrayType
from pyspark.sql.functions import *

##We use the ids in string format!, TO HAVE THE ARRAY OF STRINGS
df_book = df_book\
    .select("id","date", "url", "isbn", "citationscore", "publisher" , "title", split(col("editor_array"),",").alias("editor_array"))
df_incollection = df_incollection\
    .select("id","title","date", "url", "pages", "citationscore", \
    "book_id", split(col("author_array"),",").alias("author_array"), \
    split(col("cited_inc_array"),",").alias("cited_inc_array"))

df_incollection.printSchema()
#df_incollection.limit(5).select("id", 'author_array').show()
df_incollection = df_incollection \
  .withColumn("id" ,
              df_incollection["id"]
              .cast(StringType()))   \
  .withColumn("title"  ,
              df_incollection["title"]
              .cast(StringType())) \
  .withColumn("date",
              df_incollection["date"]
              .cast(DateType()))    \
  .withColumn("url"  ,
              df_incollection["url"]
              .cast(StringType())) \
  .withColumn("pages"  ,
              df_incollection["pages"]
              .cast(StringType())) \
  .withColumn("citationscore"  ,
              df_incollection["citationscore"]
              .cast(FloatType())) \

df_incollection.printSchema()
#df_incollection.filter(array_contains(df_incollection.author_array,"7370")).select("id","author_array").show(truncate = False)
#5371|[7371,  7370,  7372]
```

```
#df_book.printSchema()
#df_book.show()
df_book = df_book \
  .withColumn("id" ,
              df_book["id"]
              .cast(StringType()))   \
  .withColumn("isbn"  ,
              df_book["isbn"]
              .cast(StringType())) \
  .withColumn("date",
              df_book["date"]
              .cast(DateType()))    \
  .withColumn("url"  ,
              df_book["url"]
              .cast(StringType())) \
  .withColumn("publisher"  ,
              df_book["publisher"]
              .cast(StringType())) \
  .withColumn("citationscore"  ,
              df_book["citationscore"]
              .cast(FloatType())) \
  .withColumn("title"  ,
              df_book["title"]
              .cast(StringType())) \
#df_book.printSchema()

#EMAIL is already a string
#df_author.printSchema()
#df_author.limit(5).show()
df_author = df_author \
  .withColumn("id" ,
              df_author["id"]
              .cast(StringType()))   \
  .withColumn("name"  ,
              df_author["name"]
              .cast(StringType())) \
  .withColumn("citationscore",
              df_author["citationscore"]
              .cast(FloatType()))    \
#some tables with a lot of columns cannot be showed well because of jupyer lab
#the special character are supported here
```

13 | Queries and Commands

13.1. Command 1

Delete all the rows of the df_incollection that has a value that is null.

Complexity: Delete

```
#To show the difference between the deleted table instead
#of the original table I print both of them
#Delete all the rows of the df_incollection that has a value
#that is null.
df_incollection.select("id", "author_array").show(5, False)
df_incollection = df_incollection.na.drop("any")
df_incollection.select("id", "author_array").show(5, False)
```

```
+----+------------------+
|id  |author_array      |
+----+------------------+
|5367|[7367]            |
|5368|null              |
|5369|[7368]            |
|5370|[7369]            |
|5371|[7371, 7370, 7372]|
+----+------------------+
only showing top 5 rows

+----+------------------+
|id  |author_array      |
+----+------------------+
|5367|[7367]            |
|5369|[7368]            |
|5370|[7369]            |
|5371|[7371, 7370, 7372]|
|5372|[7368]            |
+----+------------------+
only showing top 5 rows

+----+-------------------------------------------------+
|id  |editor_array                                     |
+----+-------------------------------------------------+
|5067|null                                             |
|5068|null                                             |
|5069|[5180]                                           |
|5070|[5052, 5180]                                     |
|5071|[5182, 5187, 5186, 5185, 5181, 5184, 5183, 5188]|
+----+-------------------------------------------------+
only showing top 5 rows

+----+-------------------------------------------------+
|id  |editor_array                                     |
+----+-------------------------------------------------+
|5069|[5180]                                           |
|5070|[5052, 5180]                                     |
|5071|[5182, 5187, 5186, 5185, 5181, 5184, 5183, 5188]|
|5072|[4881]                                           |
|5073|[5190, 5189]                                     |
+----+-------------------------------------------------+
only showing top 5 rows
```

13.2. Command 2

This command creates add a book to the df_book, results are saved in df_final. Some queries are made to show that the book has been inserted successfully with the biggest id.

Complexity:Creation

```
columns = StructType([
    StructField('id', StringType(), False),
    StructField('date', DateType(), False),
    StructField('url', StringType(), False),
    StructField('isbn', StringType(), False),
    StructField('citationscore', FloatType(), False),
    StructField('publisher', StringType(), False),
    StructField('title', StringType(), False),
    StructField('editor_array', ArrayType(StringType()), False)
])

add_row = [['5180', datetime.strptime('2022-10-02', '%Y-%m-%d'),\
'https://www.theblackhole.com', '978-8-817-08378-2', 0.5652352,\
'Politecnico di Milano', 'The black hole', [5052, 5180]]]
add_row_df = spark.createDataFrame(add_row, columns)

df_final = df_book.union(add_row_df)
df_final.filter(col("id") == '5180').show()
df_final.orderBy(desc('id')).show()
```

```
+----+----------+--------------------+---------------+-------------+--------------------+-------------+------------+
|  id|      date|                 url|           isbn|citationscore|           publisher|        title|editor_array|
+----+----------+--------------------+---------------+-------------+--------------------+-------------+------------+
|5180|2022-10-02|https://www.thebl...|978-8-817-08378-2|    0.5652352|Politecnico di Mi...|The black hole|[5052, 5180]|
+----+----------+--------------------+---------------+-------------+--------------------+-------------+------------+

+----+----------+--------------------+---------------+-------------+--------------------+--------------------+--------------------+
|  id|      date|                 url|           isbn|citationscore|           publisher|               title|        editor_array|
+----+----------+--------------------+---------------+-------------+--------------------+--------------------+--------------------+
|5180|2022-10-02|https://www.thebl...|978-8-817-08378-2|    0.5652352|Politecnico di Mi...|      The black hole|        [5052, 5180]|
|5179|2017-05-16|https://doi.org/1...|978-0-387-09765-7|    0.9287509|           Springer|Encyclopedia of P...|              [5366]|
|5178|2020-05-30|https://doi.org/1...|978-3-642-63863-3|     1.109059|           Springer|Handbook of Forma...|        [5354, 5365]|
|5177|2018-11-25|https://doi.org/1...|978-3-642-08230-6|    0.8813321|           Springer|Handbook of Forma...|        [5354, 5365]|
|5176|2018-11-25|https://doi.org/1...|978-3-642-63859-6|    1.0178347|           Springer|Handbook of Forma...|        [5354, 5365]|
|5175|2019-02-14|https://doi.org/1...|978-1-4614-8266-6|    1.0247777|           Springer|Encyclopedia of D...|        [2017, 5364]|
|5174|2019-06-02|https://doi.org/1...|978-0-387-35544-3|   0.89383686|         Springer US|Encyclopedia of D...|        [5364, 2017]|
|5173|2022-05-04|https://www.sprin...|           null|    0.9762692|           Springer|Encyclopedia of S...|        [5363, 5362]|
|5172|2017-05-16|https://doi.org/1...|978-1-4614-6169-2|    1.1439525|           Springer|Encyclopedia of S...|        [5363, 5362]|
|5171|2011-11-25|                null|978-0-387-75888-6|    1.0866189|           Springer|Encyclopedia of C...|              [5361]|
|5170|2021-10-14|https://doi.org/1...|978-3-642-04116-7|   0.86902833|           Springer|Handbook of Infor...|        [5359, 5360]|
|5169|2015-02-09|http://www.crcnet...|978-1-4200-8592-1|  0.91111636|Chapman and Hall/CRC|Handbook of Natur...|        [5357, 5358]|
|5168|2021-04-09|https://doi.org/1...|978-3-540-92909-3|    0.8138879|           Springer|Handbook of Natur...|  [5355, 5356, 5354]|
|5167|2019-06-27|https://doi.org/1...|978-1-4939-2863-7|    1.2096438|           Springer|Encyclopedia of A...|              [5353]|
|5166|2019-06-27|https://doi.org/1...|978-0-387-30770-1|     0.870948|           Springer|Encyclopedia of A...|              [5353]|
|5165|2011-11-25|                null|978-3-540-76393-2|   0.92966974|           Springer|Taschenbuch der A...|[5348, 5350, 2363...|
|5164|2020-04-17|https://www.scien...|978-0-12-227240-0|    1.1512153|      Academic Press|Encyclopedia of I...|              [5346]|
|5163|2017-05-16|https://doi.org/1...|978-1-4471-5102-9|    0.8679281|           Springer|Encyclopedia of S...|        [5345, 5344]|
|5162|2022-03-16|https://doi.org/1...|978-3-662-53247-8|    1.1213722|           Springer|Handbuch Industri...|  [5341, 5342, 5343]|
|5161|2022-03-16|https://doi.org/1...|978-3-662-53250-8|    0.6581896|           Springer|Handbuch Industri...|  [5342, 5343, 5341]|
+----+----------+--------------------+---------------+-------------+--------------------+--------------------+--------------------+
```

13.3. Command 3

Create a new incollection

Complexity:create

```
schema = StructType([ \
    StructField("id", StringType(), True), \
    StructField("title", StringType(), True), \
    StructField("date", DateType(), True), \
    StructField("url", StringType(), True), \
    StructField("pages", StringType(), True), \
    StructField("citationscore", FloatType(), True), \
    StructField("book_id", StringType(), True), \
    StructField("author_array", ArrayType(StringType()), True), \
    StructField("cited_inc_array", ArrayType(StringType()), True) \
])

new_incollection= [("10", "Efficient View Maintenance at Data Warehouses."
    ,\
datetime.strptime("2022-09-09","%Y-%m-%d"),\
"http://dblp.uni-trier.de/papers/DiplomarbeitRitaLey.pdf",\
"456-345", 0.962399934, "5110",["2008","2128"],["5409","5410","6543"])]

df_new_inc = spark.createDataFrame(data = new_incollection, schema = schema
    )

df_new_inc.show()

df_incollection = df_incollection.union(df_new_inc)

#Check for the new insertion
df_incollection.orderBy(col("id").asc()).limit(5).show()
```

```
+---+--------------------+----------+--------------------+-------+-------------+-------+------------+--------------------+
| id|               title|      date|                 url|  pages|citationscore|book_id|author_array|     cited_inc_array|
+---+--------------------+----------+--------------------+-------+-------------+-------+------------+--------------------+
| 10|Efficient View Ma...|2022-09-09|http://dblp.uni-t...|456-345|   0.96239996|   5110|[2008, 2128]|[5409, 5410, 6543]|
+---+--------------------+----------+--------------------+-------+-------------+-------+------------+--------------------+
```

```
+----+--------------------+----------+--------------------+---------+-------------+-------+------------+--------------------+
|  id|               title|      date|                 url|    pages|citationscore|book_id|author_array|     cited_inc_array|
+----+--------------------+----------+--------------------+---------+-------------+-------+------------+--------------------+
|  10|Efficient View Ma...|2022-09-09|http://dblp.uni-t...|  456-345|   0.96239996|   5110|[2008, 2128]|  [5409, 5410, 6543]|
|5367|Ear Shape for Bio...|2017-07-12|https://doi.org/1...|  372-378|    0.8379688|   5169|      [7367]|[6264, 7146, 7146...|
|5368|      ARP Poisoning.|2017-07-12|https://doi.org/1...|       47|    0.9060627|   5067|        null|[5480, 6390, 6371...|
|5369|             Token.|2017-07-12|https://doi.org/1...|1305-1306|    0.8996254|   5109|      [7368]|[5946, 5866, 6565...|
|5370|Multi-Threaded Im...|2017-07-12|https://doi.org/1...|  823-824|    0.6736637|   5114|      [7369]|[7056, 7059, 7316...|
+----+--------------------+----------+--------------------+---------+-------------+-------+------------+--------------------+
```

13.4. Command 4

This command update the author array by adding the author with id 2000 in the incollections that have:

- citationscore greater than 1.7
- null author array

In this scenario, we are assuming that the author we are adding in the incollections is already contained in the db.

Complexity: Update

```
#author assumption
df_author.filter(col("id")=="2000").show()

#update
df_incollection.filter((col("citationscore")>1.7) &\
(col("author_array").isNull())).show()

df_incollection=df_incollection.withColumn("author_array",\
when((col("citationscore")>1.7) &(col("author_array").isNull()),\
array(lit("2000"))).otherwise(col("author_array")))

#proof
df_incollection.filter((col("citationscore")>1.7) & \
(col("author_array").isNull())).show()

df_incollection.filter(col("citationscore")>1.7)\
.sort(col("citationscore").desc()).show()
```

```
+----+-----------+-------------+--------------------+
|  id|       name|citationscore|               email|
+----+-----------+-------------+--------------------+
|2000|Paul Kocher|     0.977108|paul.kocher@gmail...|
+----+-----------+-------------+--------------------+

+----+--------------------+----------+--------------------+-----+-------------+-------+------------+--------------------+
|  id|               title|      date|                 url|pages|citationscore|book_id|author_array|     cited_inc_array|
+----+--------------------+----------+--------------------+-----+-------------+-------+------------+--------------------+
|6213|Time-Based Access...|2017-07-12|https://doi.org/1...| 1297|    1.8360946|   5130|        null|[7036, 7068, 7318...|
|6296|Public-Key Encryp...|2017-07-12|https://doi.org/1...| 1001|    2.1130114|   5128|        null|[5984, 6698, 6191...|
|6345|Fixed Window Expo...|2017-07-12|https://doi.org/1...|  482|    1.7093747|   5132|        null|[7061, 7072, 7337...|
|6841|        ML-Sequence.|2017-07-12|https://doi.org/1...|  788|    1.9255799|   5132|        null|[7364, 7321, 5720...|
+----+--------------------+----------+--------------------+-----+-------------+-------+------------+--------------------+

+---+-----+----+---+-----+-------------+-------+------------+---------------+
| id|title|date|url|pages|citationscore|book_id|author_array|cited_inc_array|
+---+-----+----+---+-----+-------------+-------+------------+---------------+
+---+-----+----+---+-----+-------------+-------+------------+---------------+

+----+--------------------+----------+--------------------+-------+-------------+-------+------------------+--------------------+
|  id|               title|      date|                 url|  pages|citationscore|book_id|      author_array|     cited_inc_array|
+----+--------------------+----------+--------------------+-------+-------------+-------+------------------+--------------------+
|6124|Public Key Proxy ...|2017-07-12|https://doi.org/1...|997-999|    2.1922653|   5149|            [7375]|[7024, 5598, 6752...|
|6296|Public-Key Encryp...|2017-07-12|https://doi.org/1...|   1001|    2.1130114|   5128|            [2000]|[5984, 6698, 6191...|
|7137|Noise in VLSI Tec...|2017-07-12|https://doi.org/1...|   null|      2.03316|   5107|[8086, 8085, 8087]|[5950, 6592, 6863...|
|6024|BLS Short Digital...|2017-07-12|https://doi.org/1...|   null|    1.9425195|   5126|            [7378]|[6997, 5491, 5509...|
|5873|Applications of F...|2017-07-12|https://doi.org/1...|  44-45|     1.940844|   5076|            [7394]|[6393, 6379, 5444...|
|6733|Karatsuba algorithm.|2017-07-12|https://doi.org/1...|   null|    1.9403228|   5092|            [7555]|[7029, 7325, 5876...|
|6841|        ML-Sequence.|2017-07-12|https://doi.org/1...|    788|    1.9255799|   5132|            [2000]|[7364, 7321, 5720...|
|6076|Conceptual Design...|2017-07-12|https://doi.org/1...|245-248|    1.8584673|   5173|      [7588, 7589]|[5609, 6066, 7188...|
|6213|Time-Based Access...|2017-07-12|https://doi.org/1...|   1297|    1.8360946|   5130|            [2000]|[7036, 7068, 7318...|
|5939|Propagation Chara...|2017-07-12|https://doi.org/1...|986-987|    1.7509822|   5118|            [7425]|[7061, 7327, 5410...|
|6345|Fixed Window Expo...|2017-07-12|https://doi.org/1...|    482|    1.7093747|   5132|            [2000]|[7061, 7072, 7337...|
+----+--------------------+----------+--------------------+-------+-------------+-------+------------------+--------------------+
```

13.5. Command 5

This command update the email address of an author called "Borko Furht"

Complexity: Update

```
df_author.filter(col("id")>4875)\
.show(10,truncate=False)

df_author.filter(col("name")=="Borko Furht")\
.show(truncate=False)

ValueWhenTrue = "B.Furht@gmail.com"
df_author=df_author.withColumn('email',when(col("name")=="Borko Furht",\
ValueWhenTrue).otherwise(col('email')))

df_author.filter(col("name")=="Borko Furht")\
.show(truncate=False)

df_author.filter(col("id")>4875)\
.show(10,truncate=False)
```

```
+----+--------------------+-------------+------------------------------+
|id  |name                |citationscore|email                         |
+----+--------------------+-------------+------------------------------+
|4878|Amir Hossein Gandomi|0.80437696   |amir.hossein.gandomi@gmail.com|
|4879|Amir Hossein Alavi  |0.80437696   |amir.hossein.alavi@gmail.com  |
|4880|Conor Ryan          |0.90832186   |conor.ryan@gmail.com          |
|4881|Borko Furht         |1.4604992    |borko.furht@gmail.com         |
|4882|Ankur Agarwal       |0.9059771    |ankur.agarwal@gmail.com       |
|4883|Ngoc Thanh Nguyen   |4.497282     |ngoc.thanh.nguyen@gmail.com   |
|4884|Ryszard Kowalczyk   |3.0994263    |ryszard.kowalczyk@gmail.com   |
|4885|Marcin Hernes       |0.71323544   |marcin.hernes@gmail.com       |
|4886|Juan M. Corchado    |1.5314459    |juan.m.corchado@gmail.com     |
|4887|Javier Bajo         |1.5314459    |javier.bajo@gmail.com         |
+----+--------------------+-------------+------------------------------+
only showing top 10 rows

+----+-----------+-------------+---------------------+
|id  |name       |citationscore|email                |
+----+-----------+-------------+---------------------+
|4881|Borko Furht|1.4604992    |borko.furht@gmail.com|
+----+-----------+-------------+---------------------+

+----+-----------+-------------+-----------------+
|id  |name       |citationscore|email            |
+----+-----------+-------------+-----------------+
|4881|Borko Furht|1.4604992    |B.Furht@gmail.com|
+----+-----------+-------------+-----------------+

+----+--------------------+-------------+------------------------------+
|id  |name                |citationscore|email                         |
+----+--------------------+-------------+------------------------------+
|4878|Amir Hossein Gandomi|0.80437696   |amir.hossein.gandomi@gmail.com|
|4879|Amir Hossein Alavi  |0.80437696   |amir.hossein.alavi@gmail.com  |
|4880|Conor Ryan          |0.90832186   |conor.ryan@gmail.com          |
|4881|Borko Furht         |1.4604992    |B.Furht@gmail.com             |
|4882|Ankur Agarwal       |0.9059771    |ankur.agarwal@gmail.com       |
|4883|Ngoc Thanh Nguyen   |4.497282     |ngoc.thanh.nguyen@gmail.com   |
|4884|Ryszard Kowalczyk   |3.0994263    |ryszard.kowalczyk@gmail.com   |
|4885|Marcin Hernes       |0.71323544   |marcin.hernes@gmail.com       |
|4886|Juan M. Corchado    |1.5314459    |juan.m.corchado@gmail.com     |
|4887|Javier Bajo         |1.5314459    |javier.bajo@gmail.com         |
+----+--------------------+-------------+------------------------------+
```

13.6. Query 1

This query returns name and pages of incollections contained in the book with the highest citationscore and an existing url as attribute. All the incollections shown in the results need to have an existing number of pages as attribute.

Complexity: WHERE, JOIN

```
df_book.filter(col("url") != "null").select("id", "citationscore")\
.filter(col("citationscore") ==df_book.groupby().max("citationscore")\
.first().asDict()['max(citationscore)']).select("id")\
.join(df_incollection.select("id", "title", "pages", "book_id"),\
(df_book.id).isin(df_incollection.book_id))\
```

```
.select("title", "pages").filter(col("pages") != "null")\
.withColumnRenamed("title","incollection_name")\
.show(truncate = False)
```

```
+-----------------------------------------------------------+---------+
|incollection_name                                          |pages    |
+-----------------------------------------------------------+---------+
|Near Field Communication.                                  |829      |
|Biometric Recognition.                                     |104      |
|FPGAs in Cryptography.                                     |499-501  |
|XML-Based Access Control Languages.                        |1395-1399|
|Fermats Little Theorem.                                    |456      |
|Chosen Plaintext Attack.                                   |205-206  |
|Worms in Cellular Networks.                                |1392-1393|
|Chip Card.                                                 |205      |
|EMV.                                                       |412-416  |
|Hash Agility.                                              |538-540  |
|Biba Integrity Model.                                      |81       |
|XSS.                                                       |1399     |
|Passport Security.                                         |913-916  |
|Shamirs Trick.                                             |1194     |
|Key Life Cycle Management.                                 |683      |
|Corporate Blogging.                                        |731-739  |
|Preparing Participants for Computer Mediated Communication.|25-33    |
|WebCom.                                                    |379-400  |
|Collaborative e-Work,e-Business,and e-Service.             |1549-1576|
+-----------------------------------------------------------+---------+
```

13.7. Query 2

Finding all the author with the name: Paul, ordering them on the citationscore and limiting the list to the top five (in the ascending order).

Complexity: WHERE, LIMIT, LIKE

```
df_author.filter(col("name").like("Paul%")).orderBy("citationscore")\
.limit(5).show(truncate = False)

#Unlike sort(), the orderBy() function guarantees a total order
#in the output. This happens because the data will be collected
#into a single executor in order to be sorted.
#This means that orderBy() is more inefficient compared to sort(),
#but more precise, so since the query is light we use orderBy.
```

```
+----+----------------+-------------+-------------------------+
|id  |name            |citationscore|email                    |
+----+----------------+-------------+-------------------------+
|7538|Paul Leyland    |0.63710874   |paul.leyland@gmail.com   |
|2359|Paul B. Hermanns|0.7432279    |paul.b.hermanns@gmail.com|
|2337|Paul Ferring    |0.8216445    |paul.ferring@gmail.com   |
|3168|Paul Firenze    |0.8230778    |paul.firenze@gmail.com   |
|7515|Paul England    |0.84346664   |paul.england@gmail.com   |
+----+----------------+-------------+-------------------------+
```

13.8. Query 3

Find the titles of incollections written after 01/01/2015 by the author "Yvo Desmedt" together with at least another author.

Complexity: WHERE, IN, Nested Query

```
author_id = df_author.filter(col("name") == "Yvo Desmedt")\
.select(col("id"))\
.collect()

author_id = author_id[0][0]

df_incollection.filter(df_incollection.date > '2015-01-01')\
.filter(array_contains(col("author_array"), author_id))\
.filter(size("author_array") > 1)\
.select(col("title"),col("date"),col("author_array"))\
.show(truncate = False)
```

```
+-----------------------------------------------------------------+----------+------------+
|title                                                            |date      |author_array|
+-----------------------------------------------------------------+----------+------------+
|Perfectly Secure Message Transmission.                           |2017-07-12|[7509, 7387]|
|Broadcast Authentication from an Information Theoretic Perspective.|2017-07-12|[7648, 7387]|
|Stream and Multicast Authentication.                             |2017-07-12|[7648, 7387]|
+-----------------------------------------------------------------+----------+------------+
```

13.9. Query 4

This query returns the number of authors of a book which are also editor of an incollection of that book. The books and incollections with null editor_array or author_array are filtered.

Complexity: GROUP BY, 1 JOIN, AS

```
#books
exploded_df = df_book.filter(col("editor_array").isNotNull())\
```

```
.select(col("id"),col("title"),explode(col("editor_array")))\
.withColumnRenamed("col", "editor")

exploded_df.limit(10).show()

#incollections
df_incol_filt=df_incollection.filter(col("author_array").isNotNull())\
.select(col("book_id"),col("id"),col("author_array"))

df_incol_filt.limit(10).show()

#join
new_df=exploded_df.join(df_incol_filt,\
exploded_df.id==df_incol_filt.book_id,"inner")\
.filter(array_contains(df_incol_filt.author_array, exploded_df.editor))

new_df.show()

new_df.groupBy("title").agg(count("title")\
.alias("Number of Author\Editor")).show()

#Alternative
#.count().withColumnRenamed("count", "Number of Author\Editor").show()
```

```
+----+--------------------+------+
|  id|               title|editor|
+----+--------------------+------+
|5069|Encyclopedia of C...|  5180|
|5070|Encyclopedia of C...|  5052|
|5070|Encyclopedia of C...|  5180|
|5071|Handbook of Ambie...|  5182|
|5071|Handbook of Ambie...|  5187|
|5071|Handbook of Ambie...|  5186|
|5071|Handbook of Ambie...|  5185|
|5071|Handbook of Ambie...|  5181|
|5071|Handbook of Ambie...|  5184|
|5071|Handbook of Ambie...|  5183|
+----+--------------------+------+

+-------+----+------------------+
|book_id|  id|      author_array|
+-------+----+------------------+
|   5169|5367|            [7367]|
|   5109|5369|            [7368]|
|   5114|5370|            [7369]|
|   5078|5371|[7371, 7370, 7372]|
|   5119|5372|            [7368]|
|   5177|5376|            [7373]|
|   5176|5377|            [7374]|
|   5131|5378|            [7375]|
|   5103|5379|            [7376]|
|   5074|5381|            [7377]|
+-------+----+------------------+

+----+--------------------+------+-------+----+------------+
|  id|               title|editor|book_id|  id|author_array|
+----+--------------------+------+-------+----+------------+
|5112|Handbook of Ellip...|  5262|   5112|5441|      [5262]|
|5071|Handbook of Ambie...|  5184|   5071|7039|[5181, 5184]|
|5071|Handbook of Ambie...|  5181|   5071|7039|[5181, 5184]|
+----+--------------------+------+-------+----+------------+

+--------------------+-----------------------+
|               title|Number of Author\Editor|
+--------------------+-----------------------+
|Handbook of Ambie...|                      2|
|Handbook of Ellip...|                      1|
+--------------------+-----------------------+
```

13.10. Query 5

This query retrieves the top 3 authors for the number of books published after 2000-02-03. To do this a filter condition is applied to the books selecting only the ones published after the chosen date and a group operation is applied to count all books of a certain author.

Complexity : WHERE, GROUP BY

```
df_book.filter(col("date")>lit("2000-02-03").cast(DateType()))\
.groupby(col("publisher"))\
.count()\
.sort(desc("count"))\
.withColumnRenamed("count","number_of_books")\
.show(3,False)
```

```
+--------------------+---------------+
|publisher           |number_of_books|
+--------------------+---------------+
|Springer            |57             |
|Chapman and Hall/CRC|18             |
|IGI Global          |9              |
+--------------------+---------------+
```

13.11. Query 6

This query returns the years when were published more than 3 incollections cited by at least 5 other incollections.

Complexity: GROUP BY, HAVING, AS

```
df_incollection\
.filter(size(col("cited_inc_array")) > 5)\
.select(year("date"))\
.withColumnRenamed("year(date)","year")\
.groupby("year")\
.count()\
.filter(col("count") > 3)\
.withColumnRenamed("count","num_incollections")\
.show(truncate = False)
```

```
+----+-----------------+
|year|num_incollections|
+----+-----------------+
|2018|25               |
|2022|17               |
|2019|28               |
|2020|15               |
|2011|86               |
|2017|1732             |
|2021|13               |
+----+-----------------+
```

13.12. Query 7

This query returns the top 4 editors (or publishers) with more publications (but at least 2), in the same day, after 2000-02-03.

Complexity: WHERE, GROUP BY, HAVING, AS...

```
df_book.filter(col("date")>lit("2000-02-03").cast(DateType()))\
.select(df_book.id.alias("BookId"), df_book.date, df_book.title, explode(
    df_book.editor_array).alias("editorId"))\
.groupby("editorId","date").count()\
.withColumnRenamed("count","numPublicationSameDay")\
.where(col("numPublicationSameDay")>2)\
```

```
.join(df_author,df_author.id==col("editorId"), "inner")\
.select("name", "numPublicationSameDay")\
.withColumnRenamed("name","Publisher Name")\
.sort(desc("numPublicationSameDay"))\
.show(4, False)
```

```
+------------------+---------------------+
|Publisher Name    |numPublicationSameDay|
+------------------+---------------------+
|Michael ten Hompel |4                    |
|Thomas Bauernhansl |4                    |
|Birgit Vogel-Heuser|4                    |
|Shoba Ranganathan  |3                    |
+------------------+---------------------+
only showing top 4 rows
```

13.13. Query 8

Finds the Ids of the best 10 books with the following steps (note that the array is grouped differently than before the unwind):

- Select the incollections which are cited at least by 25 other incollections (it is supposed that an incollection is important if it's cited a large number of times by other pubblications).
- Count for each book how many of the incollections selected with the previous query are present in it.
- Order the books using the calculated number and limit to 10 the query result.

Complexity: WHERE, Nested Query, GROUP BY

```
from pyspark.sql.functions import explode, col

# for each incollection explode the cited_inc_array
# it is obtained one column of ids
exploded_df = df_incollection.select(explode(df_incollection.\
              cited_inc_array)).withColumnRenamed("col", "cite")

# count how many times each incollection appears and discard the
# incollections cited less than 25 times
cited_ids = exploded_df.groupBy("cite")\
                       .count()\
```

```
                        .filter(col("count") > 25)\
                        .select("cite")\
                        .collect()

# convert the dataframe to an array
cited_ids = [ci[0] for ci in cited_ids]

# find the books which contains the incollections and count the number
# for each book
df_incollection.filter(col("id").isin(cited_ids))\
               .groupBy("book_id")\
               .count()\
               .orderBy(col("count").desc())\
               .limit(10)\
               .show()
```

```
+-------+-----+
|book_id|count|
+-------+-----+
|   5081|    7|
|   5067|    7|
|   5150|    7|
|   5126|    7|
|   5143|    7|
|   5110|    6|
|   5128|    5|
|   5145|    5|
|   5177|    5|
|   5132|    5|
+-------+-----+
```

13.14. Query 9

This query searches the authors that starts with "Paul", then, finds the incollections which has these authors and returns the book_id that have a total number of these(filtered authors) less than 3. The incollections with null author_array are filtered.

Complexity: WHERE, GROUP BY, HAVING, 1 JOIN

```
df_auth_filt=df_author.where(col("name").startswith("Paul"))\
.select(col("id"),col("name"))

df_auth_filt.limit(10).show()

exploded_df = df_incollection.filter(col("author_array").isNotNull())\
.select(col("id"),col("book_id"),explode(col("author_array")))\
.withColumnRenamed("col", "author").sort(col("author").asc())

```

```
exploded_df.limit(10).show()

exploded_df.join(df_auth_filt, exploded_df.author == df_auth_filt.id,\
"inner").groupBy("book_id").count()\
.withColumnRenamed("count","Number of authors")\
.where(col("count")<3).sort(col("Number of authors").desc()).show()
```

```
+----+----------------+
|  id|            name|
+----+----------------+
|2000|      Paul Kocher|
|2337|     Paul Ferring|
|2359| Paul B. Hermanns|
|2853| Paul B. Thompson|
|3123|       Paul Tubig|
|3168|     Paul Firenze|
|3211|  Paul Jones 0001|
|3244|     Paul Partlow|
|3356|    Paul Harrison|
|3698|Paul J. Laurienti|
+----+----------------+

+----+-------+------+
|  id|book_id|author|
+----+-------+------+
|7065|   5134|  2108|
|7307|   5067|  2128|
|5689|   5097|  2252|
|6028|   5124|  2252|
|7058|   5121|  3012|
|7057|   5131|  3319|
|7073|   5148|  3809|
|7042|   5067|  3842|
|5791|   5084|  4446|
|7017|   5130|  4880|
+----+-------+------+

+-------+----------------+
|book_id|Number of authors|
+-------+----------------+
|   5123|               2|
|   5111|               2|
|   5170|               1|
|   5069|               1|
|   5103|               1|
|   5119|               1|
+-------+----------------+
```

13.15. Query 10

This query retrieves the Authors with more then 50 incollections among all their books written after "2000-02-03". To do this the query considers all books written after that date; since a book can be written by multiple authors, the explode method is used to consider each author in the array (editor_array) and perform a join. Another join operation is performed to associate each incollection to its respective book. At the end a group operation is needed to collect all the incollection related to the books of the same author and count them and apply the filter condition.

Complexity : WHERE, GROUP BY, HAVING, 2 JOINs

```
df_book.filter(col("date")>lit("2000-02-03").cast(DateType()))\
.select(df_book.id.alias("BookId"), df_book.date, df_book.title ,\
explode(df_book.editor_array).alias("authorId"))\
.join(df_author, df_author.id == col("authorId"), "inner") \
.drop(col("id"))\
.select(col("BookId"),col("date").alias("PublicationDate"),\
col("title"), col("authorId"), col("name").alias("AuthorName"))\
.join(df_incollection, df_incollection.book_id == col("BookId"))\
.groupby(col("AuthorId"), col("AuthorName"))\
.count()\
.sort(desc("count"))\
.withColumnRenamed("count","number_of_incollection")\
.where(col("count")>50)\
.select(col("AuthorName"), col("authorId"),\
col("number_of_incollection")) \
.show()
```

```
+------------------+--------+----------------------+
|        AuthorName|authorId|number_of_incollection|
+------------------+--------+----------------------+
|       Borko Furht|    4881|                    76|
| Michael ten Hompel|    5343|                    73|
| Thomas Bauernhansl|    5342|                    73|
|Birgit Vogel-Heuser|    5341|                    73|
| Grzegorz Rozenberg|    5354|                    70|
|       Lior Rokach|    5220|                    58|
|      Arto Salomaa|    5365|                    54|
+------------------+--------+----------------------+
```

13.16. Query 11 (surplus)

This query first selects all the books with date > of 2000-02-03, then groups the values: title and editor_array (to then show them, this is because there can be different version of

the same book, so same name and editors), thanks to this group creates another column showing the number of publishers to filter rows with number of publishers greater than 2 and then show title, publishers and number of publishers.

Complexity: WHERE, GROUP BY, AS

```
df_book.filter(col("date")>lit("2000-02-03")\
.cast(DateType())).groupby("title","editor_array")\
.agg(size("editor_array").alias("num_of_editors"))\
.where(col("num_of_editors") > 2 )\
.withColumnRenamed("editor_array","editors")\
.show(truncate = True)
```

```
+--------------------+--------------------+--------------+
|               title|             editors|num_of_editors|
+--------------------+--------------------+--------------+
|Handbuch Industri...|  [5342, 5343, 5341]|             3|
|Encyclopedia of B...|[5326, 5323, 5324...|             4|
|Handbook of Ambie...|[5182, 5187, 5186...|             8|
|Handbuch Industri...|  [5342, 5343, 5341]|             3|
|Recommender Syste...|  [5220, 5296, 5297]|             3|
|Handbook of Heuri...|  [5328, 5327, 5313]|             3|
|Handbook of Natur...|  [5355, 5356, 5354]|             3|
|Encyclopedia of B...|[5323, 5324, 5326...|             4|
|Handbuch Industri...|  [5341, 5342, 5343]|             3|
|Handbook of Resea...|[5208, 5206, 5209...|             4|
|Handbook of Dynam...|  [5334, 5335, 5333]|             3|
|Handbook of Model...|[5211, 5210, 5213...|             4|
|Handbook on Soft ...|  [5253, 5254, 5252]|             3|
|Encyclopedia of GIS.|  [5283, 5284, 5282]|             3|
|Handbook of Ellip...|[5258, 5257, 5262...|             7|
|Handbook of Discr...|[5268, 5265, 5266...|             5|
|Handbook of Seman...|  [5336, 5337, 5033]|             3|
|Modeling and Simu...|  [5294, 5293, 5292]|             3|
|Handbuch Industri...|  [5341, 5342, 5343]|             3|
|Handbook of Signa...|[5275, 5276, 5274...|             4|
+--------------------+--------------------+--------------+
```